

Resource-Efficient Low-Rate DDoS Mitigation With Moving Target Defense in Edge Clouds

Yuyang Zhou[✉], *Member, IEEE*, Guang Cheng[✉], *Member, IEEE*, Zhi Ouyang, *Student Member, IEEE*,
and Zongyao Chen[✉], *Student Member, IEEE*

Abstract—Edge computing (EC) and container technology have been widely used to increase the flexibility of computing resources and meet the real-time requirements for delay-sensitive applications. However, it has been shown that edge clouds suffer from distributed denial-of-service (DDoS) attacks, especially low-rate DDoS (LDDoS) attacks, which can be stealthily crafted to evade detection. Unfortunately, the existing techniques cannot provide effective protection, and the amplifying resource consumption and service delay incurred by defense greatly diminish the efficiency of the security system. To tackle these problems, this paper exploits Moving Target Defense (MTD) techniques and deep reinforcement learning (DRL) for mitigating the impact of LDDoS attacks in a resource-efficient way by effectively partially invalidating, avoiding, and tolerating malicious traffic that improves the Web services' security and quality with lower overhead. We first design several lightweight MTD mechanisms by utilizing the built-in functionalities of container-based applications. To further optimize resource utilization, we formulate the interaction between attacks and MTD deployment as a Markov decision process (MDP), and adopt a deep Q-network (DQN) algorithm to achieve the best trade-off between effectiveness and overhead. The simulations prove the effectiveness of the proposed approach in LDDoS mitigation, with a significant improvement of up to 31.7% in security and 26.95% in service quality when compared with other practical strategies, and the experimental results also demonstrate that our method exhibits the lowest response time per request of 276.66 ms and the lowest webpage load time of 1.413 s with only 2.44% additional memory usage in comparison with previous works in the high workload scenario.

Index Terms—Low-rate distributed denial-of-service attacks, moving target defense, resource efficient, deep reinforcement learning, edge computing.

I. INTRODUCTION

IN RECENT years, cloud computing [1], [2] has been considered an emerging paradigm because it enables end

users to access cloud services and resources anywhere and anytime. Nevertheless, the emergence of real-time applications, especially Internet of Things (IoT) applications [3], points to the fact that cloud computing cannot solely provide efficient services for delay-sensitive requests. To deal with this problem, *edge computing* (EC) [4] is proposed to meet the real-time requirements and has been widely studied in scientific and commercial fields of IoT [5]. Compared to traditional cloud computing, EC has the advantage of lower transmission delay, as the computation tasks are transferred from the remote cloud center to edge clouds [6], which process data in the proximity of IoT devices without requiring a large amount of data exchange [7].

As the key to the success of cloud computing, virtualization undoubtedly enhances the flexibility and utility of underlying hardware, and provides stability in aspects of cost and resource efficiency [8]. Due to the latest advances in lightweight virtualization techniques, *containers* [9] have been rapidly developed and widely used, especially in microservices, IoT, and EC, due to fewer resources and flexible deployment [10]. The combination of EC and containers will provide the potential to develop a wide range of new services and applications [11], hence bringing innovation and driving new businesses.

Despite the advantages and broad applications of edge computing, the number of security problems [12], [13], as well as the difficulty in solving these issues in EC-assisted applications, has dramatically increased with the continuous expansion of scale [14]. The rapid growth in IoT devices, coupled with weak security protection, has promoted the IoT-based *Distributed Denial-of-Service* (DDoS) attack [15], [16], [17], which remains one of the most challenging areas of research for the EC environment. Recent research has shown that advanced attackers could elaborately mimic the patterns of legitimate users to construct malicious requests [18] and launch a DDoS attack in a stealthy fashion, which is being called the *Low-rate DDoS* (LDDoS) attack [19].

Due to the strong concealment and destructiveness [20], LDDoS attacks have the potential to produce more damage (e.g., service interruption, service quality degradation, etc.) without suitable detection and effective defense methods. Therefore, LDDoS mitigation has become a crucial challenge, and it attracted researchers to propose methods that can effectively defeat stealthy attacks. Most of the existing DDoS mitigation methods (e.g., firewall [21], intrusion detection system [22], traffic redirection [23], etc.) rely on prior knowledge about the attack characteristics, however, these

Manuscript received 29 May 2023; revised 22 April 2024 and 24 May 2024; accepted 9 June 2024. Date of publication 13 June 2024; date of current version 14 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant No. 62202097 and Grant No. 62072100, in part by the China Postdoctoral Science Foundation under Grant No. 2024T170143 and Grant No. 2022M710677, in part by the Jiangsu Funding Program for Excellent Postdoctoral Talent under Grant No. 2022ZB137, and in part by the Southeast University-China Mobile Research Institute Joint Innovation Foundation under Grant No. CMYJY-202100163. The associate editor coordinating the review of this article and approving it for publication was N. Zincir-Heywood. (*Corresponding author: Guang Cheng.*)

The authors are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China, also with Purple Mountain Laboratories, Nanjing 211111, China, and also with the Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing 211189, China (e-mail: yyzhou@seu.edu.cn; chengguang@seu.edu.cn; oyzhi@seu.edu.cn; solar1s@seu.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3413685

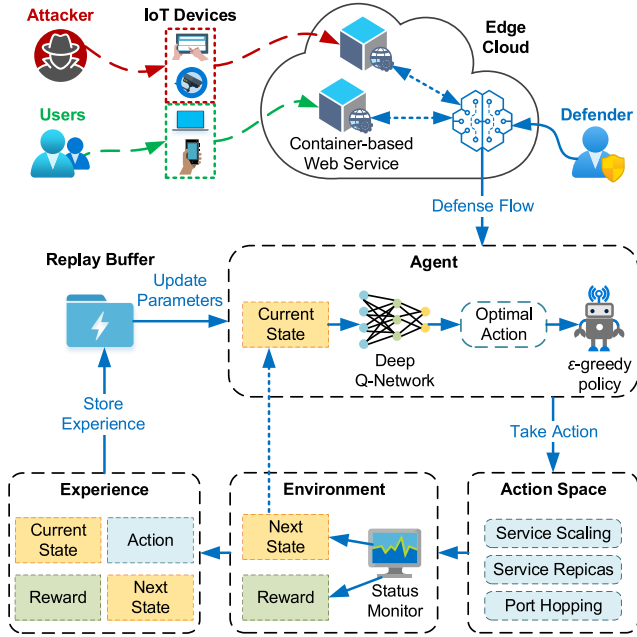


Fig. 1. Bird's-eye view of the proposed LDDoS mitigation approach.

methods suffer from efficiency and accuracy, which could be insufficient and hysteretic in the LDDoS attack scenarios. This prompts a need for proactive protection of the EC environment to effectively defeat LDDoS attacks.

In the last few years, *Moving Target Defense* (MTD) has emerged as a promising proactive solution for DDoS mitigation [24], [25]. It aims to defend against cyberattacks in a proactive manner by introducing uncertainty and unpredictability on the attacker side [26], [27]. MTD-based methods usually modify the attack surface of the protected system, to reduce the opportunity of being exploited by attackers, forcing them to continually investigate the target system and increasing their efforts to launch a successful attack [28].

However, there still exist several drawbacks in these MTD-based studies. Prior work has been proposed against flooding attacks, which cannot be directly applied to LDDoS mitigation due to the considerable differences in attack mechanisms [29]. Also, existing methods suffer from a lack of strategic guidance for defense deployment, thus strong attackers may bypass MTD techniques through long-term exploration [30] and wage attacks successfully. Furthermore, frequent MTD adaptations (e.g., network reconfiguration, virtual machine (VM) migration) may have negative effects on the EC environment by reducing the quality of service (QoS) on top of high overhead [31], which could be improved. These reasons motivate us to achieve an effective balance between resource use and security improvement with lightweight MTD mechanisms and optimized strategies.

To address the aforementioned challenges, we propose a novel LDDoS mitigation method, namely *RE-MTD*, for preventing LDDoS attacks by establishing and deploying MTD techniques in a resource-efficient way. Fig. 1 shows the bird's-eye view of the proposed method. The key insight of this work is to develop ever-changing network properties, thus

shifting the attack surface to confuse attackers. Specifically, we establish lightweight MTD mechanisms based on the built-in functionalities of the containers, to force attackers to continuously chase the target. In order to achieve resource-efficient defense, we exploit *deep reinforcement learning* (DRL) [32] to develop an adaptive and intelligent strategy that dynamically mitigates the QoS degradation caused by LDDoS attacks within limited edge resources. We first formulate the interaction between attack and defense deployment as a *Markov decision process* (MDP). Then, we adopt a *deep Q-network* (DQN) algorithm to obtain the best trade-off between resource use and effectiveness. Finally, we conduct a set of extensive simulations and experiments to demonstrate the effectiveness of our method. The main contributions of this paper are summarized as follows:

- We present a novel LDDoS mitigation approach, namely RE-MTD, to ensure enhanced security and acceptable overhead by deeply exploiting MTD and DRL techniques. To the best of our knowledge, this work is the first MTD-based solution for defeating hard-to-detect LDDoS attacks in a resource-efficient manner.
- We propose novel MTD mechanisms specifically designed for the EC environment by leveraging the intrinsic characteristics of containers. These lightweight and flexible mechanisms can be developed and deployed rapidly, and are effective in collaboratively mitigating LDDoS attacks across a wide range of attack strengths and scales.
- We introduce a DRL-based framework to facilitate defense deployment within limited resource constraints. We formulate the interaction between the LDDoS attacks and the MTD mechanisms as an MDP, and we utilize a DQN algorithm to solve the optimal policy, which allows us to achieve resource-efficient defense by balancing resource consumption and mitigation effectiveness.
- We evaluate our method through simulations and real-world experiments. The results demonstrate that RE-MTD can effectively mitigate LDDoS attacks, even in resource-constrained environments. Furthermore, RE-MTD outperforms the state-of-the-art (SOTA) methods in terms of both QoS and security, while imposing less overhead.

The remainder of this paper is organized as follows. We review the related work in Section II, and we provide the threat model in Section III. Section IV demonstrates the details of MTD mechanisms and Section V introduces the DRL-based strategy. Section VI presents the evaluation results. We discuss the limitations of our work and future challenges in Section VII. Finally, we conclude in Section VIII. All the notations used in this paper are shown in Table I.

II. BACKGROUND AND RELATED WORK

A. LDDoS Attacks in the EC Environment

DDoS attacks have always been a huge threat [14], and attackers have been modifying their attack format to evade detection [33]. In recent years, LDDoS has emerged as a sophisticated attack [34]. This attack exploits specific

TABLE I
LIST OF TERMINOLOGIES WITH THEIR EXPLANATORY TERMS

Notation	Description	Notation	Description
$\mathcal{N}$	Set of services	R_t^q	Reward of service quality
T	Time steps	$\tilde{c}_n(t)$	Load rate
S_t	State space	R_t^s	Reward of security
$r_n(t)$	Running status	$\phi_n(t)$	Security indicator
$c_n(t)$	Number of connections	R_t^c	Reward of resources
$e_n(t)$	Resource consumption	R_t^o	Reward of time overhead
$p_n(t)$	Port number	π	Optimal strategy
A_t	Action space	$V^\pi(s)$	Value function
$\tilde{A}_t^{sca}$	Service scaling	$Q^\pi(s, a)$	Q-value function
$\tilde{A}_t^{rep}$	Service replica	L	Loss function
$\tilde{A}_t^{hop}$	Port hopping	ω	Model parameter
$P(s' s, a)$	Transition probability	B	Replay buffer
R_t	Reward function	C	Update period

vulnerabilities through semantic methods and sends malicious requests at a low frequency [35].

According to [36], LDDoS attacks can be divided into three categories: The first is QoS attack that aims to deteriorate the QoS of the victim. For example, in Shrew attack [37], the attacker launches attack pulses based on the Retransmission Timeout (RTO) mechanism of TCP sender [38], shutting off most of the legitimate TCP sources and greatly diminishing the QoS of normal users. The second is Slow DDoS attack which targets Hyper Text Transfer Protocol (HTTP) servers [39]. It occupies hundreds of HTTP connections and prevents it from processing further requests, significantly increasing webpage loading time or rendering it inaccessible [40]. The third type is Service Queue attack that directly affects the data processing queue of the services. For example, the Low-Rate DDoS attacks against Application Servers (LoRDAS) [41] repeatedly insert malicious requests to occupy the service queue, so the server cannot process other incoming packets.

We mainly focus on Slow DDoS mitigation in this paper due to the development of IoT applications, especially mobile Web applications in the EC environment.

B. LDDoS Mitigations in the EC Environment

Traditional DDoS detection methods [22], [42], [43] utilize attack patterns, such as byte or packet sequence, to identify malicious traffic using Machine Learning (ML) or Deep Learning (DL) algorithms. Nevertheless, they may fail due to the absence of typical features in LDDoS attacks.

To cope with this problem, researchers have presented other mitigations against these attacks. For instance, Wu et al. [44] explored the multifractal characteristics over a small scale of network traffic to detect attack traffic by the algorithm of multifractal detrended fluctuation analysis (MF-DFA). Nevertheless, as attackers become aware of the detection rules, the accuracy will continue to decrease. Tang et al. [45] proposed an improved adaptive Kohonen Network clustering algorithm for TCP-targeted attack detection, which can detect attack bursts and estimate the beginning and end times of attacks. However, the clustering-based method may misclassify attack traffic as benign. Besides, they also proposed LtRFT [46], a Learning-To-Rank (LtR) based scheme for mitigating LDDoS attacks targeted at flow tables in

Software-Defined Networking (SDN) environment. However, this method cannot be applied to non-SDN scenarios. With the formulation of the Slow DDoS attack scenario by queueing theory, Li et al. [19] regulated the number of containers for different users and proposed an attack mitigation method based on resource allocation, but it lacks deployment guidance with an optimized strategy. Fu et al. [47] utilized sequential information to build the frequency domain features and proposed Whisper to detect malicious traffic in real time. It was resistant to some attacks, but the feature extraction and construction operations are consuming considerable costs. Tripathi and Hubballi [41] utilized the Chi-Square distance between the legitimate and attack traffic for the detection of LoRDAS attacks against HTTP/2. In addition, an event sequence analysis-based scheme was proposed by Tripathi [48] to detect attacks with a real-time strategy, however, it cannot maintain high accuracy once the characteristics of attack traffic change.

C. MTD-Based Solutions

Recently, a series of MTD-based methods have been proposed to defeat high-rate DDoS attacks in the cloud environment. The main goal of these solutions is to force attackers to chase the wrong targets with higher efforts and costs, thereby keeping DDoS attacks away from the protected system.

Debroy et al. [49] mitigated DDoS attacks by optimizing the frequency of VM migrations in the SDN-enabled cloud environment. However, this method suffers from heavy overhead. In a similar scenario, Zhou et al. [24] proposed an MTD framework based on adaptive network configurations and a signaling game model to improve the effectiveness. Although this scheme optimizes the defense strategy based on game theory, it may be ineffective against stealthy low-rate attacks.

To achieve lightweight DDoS defense in the EC environment, Zhou et al. [50] adopted container migrations and utilized the Reinforcement Learning (RL) method to optimize defense strategies, however, it does not have multiple defense mechanisms or achieve a trade-off between effectiveness and resources. Similarly, Zhang et al. [25] modeled the configuration mutation as a Markov decision process and defeated DDoS attacks based on the solver of optimal network configurations. They also proposed a collaborative mutation-based MTD (CM-MTD) method [51], which dynamically adjusts network properties and invalidates different stages of the cyber kill chain with DRL-based guidance. Nevertheless, these methods are equipped with high computation resources. In addition, Osei et al. [52] formulated an optimization problem using a game-theoretic model for DDoS attack mitigation and Kim et al. [32] exploited deep reinforcement learning to build a moving target defense countermeasure framework. Although these schemes protect network systems against DDoS attacks, it does not guarantee the QoS of the deployed services.

Table II entails a comprehensive feature analysis and comparison of RE-MTD with state-of-the-art works. RE-MTD is the individual approach that supports resource-limited EC environment and lightweight defense deployments by

TABLE II
FEATURES COMPARISON BETWEEN RE-MTD AND STATE-OF-THE-ARTS WORKS

Scheme	Characteristics								Methods		
	AS	PT	DM	LT	PD	LD	SO	RE	Basis	Classifier	Security
CFS-BA [22]	***	*	*	×	×	×	×	×	Network Traffic	ML	Detection
Flowguard [42]	***	*	*	×	×	×	×	×	Network Traffic	DL	Detection
Fleam [43]	***	***	***	×	×	×	×	×	Network Traffic	FL	Detection & Prevention
MF-DFA [44]	*	*	*	✓	×	×	×	×	Network Traffic	×	Detection
AKN-FGD [45]	*	*	*	✓	×	×	×	×	Network Traffic	Clustering	Detection
LiRFT [46]	***	*	***	✓	×	×	✓	×	Network Traffic	ML	Detection & Prevention
Queueing Theory [19]	***	***	*	✓	×	✓	×	×	System Status	×	Prevention
Whisper [47]	*	***	*	✓	×	×	✓	×	Network Traffic	Clustering	Detection
Chi-Square Distance [41]	***	*	*	✓	×	×	×	×	Network Traffic	×	Detection
Event Sequence [48]	***	*	*	✓	×	×	×	×	Network Traffic	×	Detection
FM-UM [49]	***	***	*	×	✓	×	✓	×	System Status	×	Prevention
OSA [24]	***	***	***	×	✓	×	✓	×	System Status	×	Prevention
CMDP-MOS [50]	***	***	*	×	✓	✓	✓	×	Network Status	RL	Prevention
PPO-OCM [25]	***	***	*	×	✓	×	✓	×	Network Status	DRL	Prevention
CM-MTD [51]	***	***	***	×	✓	×	✓	×	Network Status	DRL	Prevention
Optimized MTD [52]	***	***	*	×	✓	×	✓	×	Network Status	×	Prevention
DIVERGENCE [25]	*	*	***	×	✓	×	✓	×	Network Status	DRL	Detection & Prevention
RE-MTD (Proposed)	***	***	***	✓	✓	✓	✓	✓	Service Status	DRL	Prevention & Toleration

*: Single; ***: Multiple; AS: Attack Source; PT: Protected Target; DM: Defense Mechanism; LT: Low-rate attack Targeted; PD: Proactive Defense; LD: Lightweight Deployment; SO: Strategy Optimization; RE: Resource Efficient; FL: Federated Learning.

leveraging the inner characteristics of containers as MTD mechanisms, and exploits the DRL algorithm to optimize effectiveness against LDDoS attacks while reducing unnecessary overhead. The major drawback of the existing works is that the static and deterministic patterns provide opportunities for stealthy attackers to evade malicious traffic detection, resulting in low accuracy and limited effectiveness. Also, the previous MTD-based solutions were primarily designed to mitigate high-rate flooding attacks with high economic costs and resource consumption. Unlike the earlier works, RE-MTD establishes a proactive LDDoS attack-targeted method to prevent and tolerate malicious requests without relying on attack patterns. The proposed method enables multiple edge servers to achieve resource-efficient defense while considering a wide range of attack strengths and scales.

III. THREAT MODEL

In our investigation, we consider an edge cloud in which exists a couple of containers to provide Web services for legitimate users. We assume that the key infrastructure and the service provider are trusted, while the attackers come from an external network. To perform a comprehensive analysis and develop effective countermeasures against these attacks, we consider a threat model in terms of explicit assumptions about the attack goal, implementation process, and capabilities the attackers have.

A. Attack Goal

In general, attackers exploit the inherent vulnerabilities of HTTP protocols and stealthily send attack requests to fill the connection pool of container-based Web services. Consequently, legitimate users' HTTP connection attempts would be denied, thereby downgrading their experiences or further causing network paralysis.

Especially, we consider a rational and tactful attacker who aims to maximize his/her damage to the EC environment in this study. That is, the attacker will amplify the attack effect as much as possible by carrying out LDDoS attacks based on the controlled IoT devices, maximizing the number of compromised service replicas or legitimate users that suffer from these attacks.

B. Attack Process

To formally depict the details of the attack process, we assume that the attacker performs a multi-round attack by following the Cyber Kill Chain (CKC), from early reconnaissance to the actual attack, as follows:

- The attackers first utilize scan tools (e.g., Nmap and ZMap) to perform reconnaissance on the edge cloud, but may only obtain short-term and fragmentary results due to the existence of MTD mechanisms.
- According to their existing knowledge and experience, attackers assess the impact they will have on potential victims prior to taking actions, and select appropriate container instances as attack targets.
- The attackers then carry out LDDoS attacks on these container-based Web services to accomplish their destructive goals via cyber weapons (e.g., SlowHTTPTest¹ and RU-Dead-Yet²).
- Following that, they evaluate the payoffs obtained from launching attacks and move on to the next round of reconnaissance.

C. Attacker's Capabilities

The key to fighting a DDoS attack is resource competition, the defender will never cope with it if the attacker owns unlimited resources. Therefore, we focus on an attacker who

¹<https://github.com/shekyaan/slowhttptest>

²<https://sourceforge.net/projects/r-u-dead-yet/>

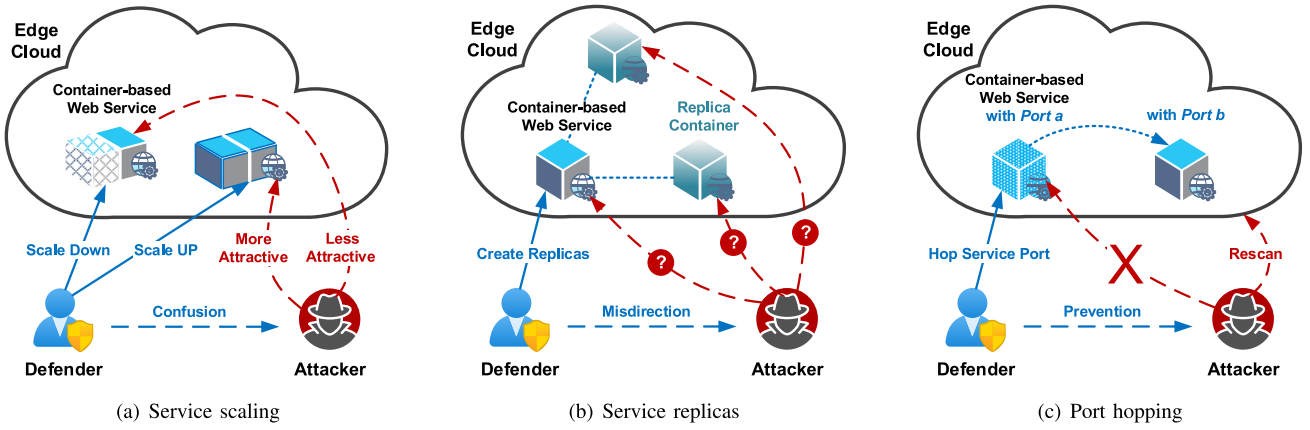


Fig. 2. An overview of the proposed MTD-based mitigation mechanisms, including (a) service scaling, (b) service replicas, and (c) port hopping.

controls a botnet that consists of a certain number of IoT devices, and we scope his/her capabilities as follows:

- We assume that the attacker can wage LDDoS attacks in a stealthy fashion and has the ability to evade detection at the very beginning of attacks.
- Given the limited resources that the attacker can afford, he/she cannot compromise all the serving containers among the edge cloud.
- According to the information collected during the reconnaissance phase, all attackers will strategically choose targets to maximize their attack effect.
- Due to the different characteristics of attackers, their behavior patterns will also be different. A cursory attacker's actions may be immutable during the attack, while the actions of a tactful attacker may change based on the continuous observation of the victims.

For the sake of not weakening the attacker's ability, we assume that he/she may know in advance that there exist some defensive methods but not their specific deployment. In addition, tactful attackers have the ability to adjust actions or turn to another target, however, they may insist on taking the same action on the initial target due to reconnaissance results and comprehensive judgment.

IV. MTD-BASED MITIGATION MECHANISMS

In order to compromise the system from the external network, the attacker has to identify potential targets and explore weaknesses that can be exploited during the reconnaissance phase. To weaken the attacker's inherent time and cost advantages, MTD has emerged as a proactive defense mechanism that breaks the static nature by dynamically moving the attack surface and creating uncertainty on the attacker's side. The existing MTD techniques have demonstrated their effectiveness in greatly enhancing network resilience and effectively protecting the system from attacks [53].

Due to the capabilities of elastic capacity provisioned in the cloud, a lot of MTD techniques have been implemented to defeat high-rate DDoS attacks, such as IP address randomization, topology mutation, and VM migration. Regardless of the effectiveness of these solutions, the high resource consumption makes them unsuitable for resource-constrained

edge computing scenarios, and the accompanying high latency cannot meet the real-time requirements of IoT applications.

To overcome the aforementioned limitations of existing work, we propose a few container-based mitigation methods with lightweight properties, which can be flexibly deployed with a small overhead. Specifically, we leverage dynamicity, redundancy, and variability to design these MTD mechanisms corresponding to the characteristics of Slow DDoS attacks, to effectively maintain service availability and optimize resource consumption. Fig. 2 shows an overview of the proposed MTD-based mitigation mechanisms.

A. Service Scaling

As we discussed in Section III, LDDoS attackers first craft and send a massive number of requests. Then, they exploit HTTP behavior to keep the request connections open by sending each request body very slowly. In this case, HTTP requests from legitimate users cannot be received and processed by Web services. According to the characteristics of filling HTTP connections by the Slow DDoS attack, we exploit the dynamicity in the context of MTD to utilize service scaling for dynamically defeating it.

The scaling of service enables service providers to consistently increase or decrease the desired computing and storage resources of container-based applications, according to resources needed to handle all incoming requests or events. Despite the fact that it is primarily an operation and maintenance technology, fluctuation in resources caused by service scaling will directly affect the maximum number of supported connections. When the defender scales up one of the running Web services, it will be able to allow more parallel HTTP connections. In contrast, the number of connections the container can support will decrease as it has been scaled down to release some resources.

Specifically, when the maximum number of connections exceeds the capacity that an attacker can occupy (the attacker owns limited resources), it is possible for the container to process additional requests from legitimate users, thereby ending the current LDDoS attack. Conversely, we can scale down some idle containers to release unnecessary resources if there is no attack. This feature enables us to flexibly

manipulate the resource consumption of the protected service and achieve efficient defense by strategically scaling the container. In addition, since service scaling has been built into many container management systems (e.g., Kubernetes, Docker Swarm, etc.), we can easily deploy this mechanism in the EC environment, and exploit it to implement moving target defense without deteriorating the QoS of the protected application.

B. Service Replicas

Generally, a service replica is a copy of the running service created from the template of the primary container-based service. This technology fulfills the service provider's purpose by creating and deleting replicas on demand to reach the desired number at any given time. In the proposed method, replicas can be utilized to effectively release the negative impact of malicious requests on the container due to its redundancy to provide additional services.

As an example, we can create a replica of the high-load (or compromised) Web service, and redirect half of the currently active connections to the new container, thus generating a connection pool with about half the load. After that, the upcoming requests can be processed as long as the attacker is unable to fill the enlarged connection pool further. Even if the powerful attacker can increase its attack strength to open new connections, we can also mitigate the attack by creating multiple replicas. As another example, it is possible to reduce the number of running replicas, in the case of little attacking firepower and a large amount of idle resources, effectively reducing resource consumption without affecting defense performance.

Similarly, there have been corresponding technologies in the container management systems that can help us quickly implement this MTD mechanism. For example, an application deployment is usually composed of a set of pods in Kubernetes, therefore, we can achieve service replication by deploying multiple identical pods and routing different requests to one of the corresponding pods [54].

C. Port Hopping

Despite the capability of the above MTD mechanisms to mitigate LDDoS attacks via resource scheduling, there is still a chance that the radical attacker might concentrate all their firepower to strike a specific target, leaving computing resources unavailable for resisting attacks in the action phase. Thus, it is necessary to design another MTD mechanism to prevent the attack from causing further damage when it occurs in the event of limited resources.

Unlike MTD mechanisms that rely on resource scheduling, hopping-based mechanisms should move an entry involved in the attack chain so that the LDDoS attack can be stopped immediately. Although the existing methods can certainly mitigate attacks, there are some unavoidable drawbacks in part of them. For example, IP address randomization is useful in preventing unauthorized attackers from achieving network reconnaissance, but its deployment usually relies on SDN architecture [55] or virtual-real IP address translation [56].

This implementation of an SDN controller may bring extra overhead and result in a single point of failure, while attackers could break through the address translation once they receive real IP addresses. Therefore, to mitigate the attack efficiently without introducing additional risks, we adopt the method of *port hopping* as the MTD mechanism to prevent attack traffic from reaching the target.

In particular, the modified port number can be generated using a hash function (e.g., SHA-256) with a shared session key among legitimate users [57]. Besides, the Real-Time Clock (RTC) is strictly synchronized via an external server with the Network Time Protocol (NTP) [58]. In the container-based EC environment, the port hopping mechanism can be realized in a lightweight way (e.g., modification of configuration files via the APIs of the management platform), to dynamically change the port number of a particular service. As a result, the LDDoS attacker will lose the information of the entry point when it works, and will have to spend extra time and effort to determine the new port number after the move. Fortunately, with the help of the Web cache, the problem of dropping existing connections caused by this movement will not be easily perceived by users, and we discuss the time overhead caused by port hopping in the reward function in Section V-A4.

V. DRL-BASED OPTIMAL MTD DEPLOYMENT

Despite the effectiveness brought by MTD mechanisms, unreasonable deployment will weaken the effect and incur additional resource consumption. Therefore, it is necessary to optimize strategies for resource-efficient defense. As expected, we aim to improve the resilience and survivability of protected services against LDDoS attacks within the constraints of limited edge resources. At the same time, according to the different characteristics of the proposed MTD mechanisms, fair and reasonable selection of them in different attack scenarios is also essential in decision-making [59]. Due to the advantages of dealing with continuous control problems, especially in dynamic situations, we adopt the method of DRL to solve the optimization problem of MTD deployment.

A. Problem Formulation and Environment Modeling

In this section, we first consider an edge cloud that provides multiple container-based services for legitimate users. Then, we formulate the interaction between LDDoS attacks and the MTD deployment as an MDP problem, which can be completely described through its state space, action space, transition probabilities, and reward function as follows.

1) *State Space*: We denote the set of services deployed in the edge cloud by $\mathbb{N}$, written as $\mathbb{N} = \{1, 2, \dots, N\}$, where N indicates the maximum number of services that can be accommodated. Also, we consider MTD operations in discrete time steps $\mathbb{T} = \{1, 2, \dots, T\}$, where T denotes the time horizon. According to the basic idea of MDP, the system state S_t is only related to the state and action of the previous time step.

In our scenario, the intelligent agent is responsible for observing the state of every service at the beginning of each

time step, which can be represented as

$$s_n(t) = \{r_n(t), c_n(t), e_n(t), p_n(t)\}, \quad n \in \mathbb{N}, \quad (1)$$

where $r_n(t)$ is a binary value to indicate the running status for the n th service, $c_n(t)$ denotes the number of current connections, and $e_n(t)$ represents the resource consumption for computation and storage. In addition, $p_n(t)$ is the port number at this time step. Thus, we can obtain that $S_t = \{s_1(t), s_2(t), \dots, s_N(t)\}$, where $t \in \mathbb{T}$.

2) *Action Space*: We assume that the agent makes defense decisions in an independent manner, implying that it may decide to take different actions on specific services. An intelligent agent can execute one of three MTD mechanisms as described in Section IV. Hence, we can define the action space for any state in S_t as

$$A_t = [\tilde{A}_t^{sca}, \tilde{A}_t^{rep}, \tilde{A}_t^{hop}], \quad t \in \mathbb{T}, \quad (2)$$

Here, the action space is discrete. Specifically, $\tilde{A}_t^{sca} = \{a_{t,1}^{sca}, a_{t,2}^{sca}, \dots, a_{t,N}^{sca}\}$ denotes the decisions of service scaling at time step t , where $a_{t,n}^{sca} = 1$ represents the capacity expansion of the n -th service whereas $a_{t,n}^{sca} = -1$ means scaling down this service. Similarly, $\tilde{A}_t^{rep} = \{a_{t,1}^{rep}, a_{t,2}^{rep}, \dots, a_{t,N}^{rep}\}$ denotes the actions of service replicas for all deployed services, where $a_{t,n}^{rep} = 1$ denotes the creation of a copy for the n -th service whereas $a_{t,n}^{rep} = -1$ represents deleting a replica. Furthermore, $\tilde{A}_t^{hop} = \{a_{t,1}^{hop}, a_{t,2}^{hop}, \dots, a_{t,N}^{hop}\}$ shows a series of port hopping decisions on this edge cloud, where $a_{t,n}^{hop} = 1$ triggers the change of the port number for the given service. It is worth noting that $a_{t,n}^* = 0$ indicates no action, that is, this MTD mechanism will not be implemented on this service at this time.

3) *Transition Probabilities*: Let $S_t \times A_t \times S_{t+1} \rightarrow [0, 1]$ denote the state transition function. Hence, $P(s'|s, a)$ is the transition probability to s' at the next time step, given that the current state S_t is s while the agent chooses action $a \in A_t$ in this time step. This probability depends on the state of the system, the way the attacker wage LDDoS attacks, and the actions taken by the defender. Especially, we will obtain a binary transition probability for deterministic transition, which can be expressed as

$$P(s'|s, a) \in \{0, 1\}, \quad \forall s, s' \in S_{t+1}, a \in A_t, \quad (3)$$

For example, if the defender decides to remain static while the attacker maintains current malicious connections, then the system state will not have a transition and $P(s'|s, a) = 0$, where a is an array of all zeros and $s' \neq s$. Another example is the decision to change the port number in the absence of attacks, which can directly determine the following system state.

4) *Reward Function*: In MDP, the reward function motivates an agent to take appropriate actions in each state to help the agent reach the optimum policy [60]. Bear in mind that our goal is to maximize the resource efficiency of proposed MTD mechanisms against LDDoS attacks. Thus, when a defensive action is selected at time step t , we define the reward R_t with four key factors, including service quality fluctuations,

container security status, additional deployment resources, and time cost, which can be represented as

$$R_t = \alpha R_t^q + \beta R_t^s - \zeta R_t^e - \eta R_t^c, \quad t \in \mathbb{T}, \quad (4)$$

where α, β, ζ , and η are coefficients for balancing the reward. First, the reward of service quality fluctuations is defined by $R_t^q = \frac{\sum_{n=1}^N \Delta \check{c}_n(t)}{\sum_{n=1}^N r_n(t)}$, where $\Delta \check{c}_n(t) \in (-1, 1)$ denotes the change in the load rate (the ratio of the current number of connections to the upper limit). Second, the reward of container security status is defined by $R_t^s = \frac{\sum_{n=1}^N \phi_n(t) r_n(t)}{\sum_{n=1}^N r_n(t)}$, where $\phi_n(t)$ is an indicator that represents whether this container is in a secure state, if so, its value is assigned as 1, otherwise its value is assigned as 0. Then, we can define the reward of additional deployment resources as $R_t^e = \sum_{n=1}^N \Delta e_n(t)$, where $\Delta e_n(t)$ represents the difference in the deployment resource consumption at time step t . Finally, the reward of time overhead is defined as the number of hopping services, which can be expressed as $R_t^c = \sum_{n=1}^N a_{t,n}^{hop}$.

Thus, the first part of the reward function in Eq. (4) captures the number of requests made to the Web services, providing motivation for the intelligent agent to execute appropriate MTD mechanisms. It aims to minimize the impact of malicious connections on normal requests and enhance the overall QoS for legitimate users. The second part is an indicator that reflects the security status of the specific service. It drives the agent to choose a strategy at each step to strive to prevent malicious traffic from reaching the victim and maintain system security. Furthermore, the implementation of MTD mechanisms necessitates additional computing resources and introduces time overhead. Consequently, the objective of the third part is to minimize the utilization of defensive resources beyond the service requirements. Additionally, the fourth part emphasizes the reduction of unnecessary time overhead through the strategic planning of the port hopping mechanism.

5) *Convergence*: The proposed MTD-based method aims to learn a strategy π for maximizing cumulative rewards, which can optimize resilience to LDDoS attacks and resource utilization in the EC environment. Therefore, we define it as the long-term value, thereby the value function V^π can be expressed as

$$V^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=1}^T \gamma^{t-1} R(t|S_t = s) \right], \quad (5)$$

where $\mathbb{E}()$ is the expectation operator and $\gamma \in (0, 1)$ is a discount factor that ensures the agent considers the future consequences of making an action at each state. It can be also shown by manipulating the reward series in Eq. (5) that the long-term value satisfies the Bellman equation [61] as

$$V^\pi(s) = R(s, a) + \gamma \mathbb{E}[V^\pi(s')], \quad (6)$$

where $R(s, a)$ is the immediate reward received after taking action a in state s and $V^\pi(s')$ is the value function for the successor state s' .

Theorem 1: Given a finite state and action space MDP, for every policy π and state s , we can have $V^\pi(s) \leq V^*(s)$.

Proof: Suppose there exists a policy π that satisfies $V^\pi(s) > V^*(s)$ for some states. Consider taking the optimal policy π^* at state s , $V^*(s)$ can be expressed as the maximum expected return over all possible actions such that

$$V_1^*(s) = \max_{a \in A_t} (R(s, a) + \gamma \mathbb{E}[V_1^*(s')]), \quad (7)$$

Based on our assumption, $V^\pi(s) > V^*(s)$, thus

$$V^\pi(s) > \max_{a \in A_t} (R(s, a) + \gamma \mathbb{E}[V^*(s')]), \quad (8)$$

Since the value function $V^\pi(s)$ when taking the policy π can be computed based on Eq. (6), we can deduce that

$$R(s, a) + \gamma \mathbb{E}[V^\pi(s')] > \max_{a \in A_t} (R(s, a) + \gamma \mathbb{E}[V^*(s')]), \quad (9)$$

This inequality implies that there must exist an action whose value function is greater than the maximum expected return obtained by taking the action at state s according to the optimal policy π^* . In other words, policy π is yielding a higher expected return at state s compared to the optimal action prescribed by π^* , which means that we can improve the policy π^* by choosing a different action a at state s . However, this contradicts the definition of the optimal policy, which is designed to maximize the expected return at each state.

Therefore, the assumption that $V^\pi(s) > V^*(s)$ for some state s cannot hold, and we can conclude that the value function of any policy is less than or equal to the return of the optimal policy, i.e., $V^\pi(s) \leq V^*(s)$ for all states s . ■

Theorem 2: Given a finite state and action space MDP, there exists a unique optimal value function $V^*(s)$.

Proof: Assume that there exist two distinct optimal value functions V_1^* and V_2^* for any state s . Considering the difference between the two optimal value functions, we can describe it as

$$D(s) = |V_1^*(s) - V_2^*(s)|, \quad (10)$$

By substituting $V_1^*(s)$ and $V_2^*(s)$ into the Bellman optimality equation, we can deduce that

$$D(s) \leq \gamma \mathbb{E}[|V_1^*(s') - V_2^*(s')|] = \gamma \mathbb{E}[D(s')], \quad (11)$$

Using the contraction mapping principle [62], the distance between two value functions should reduce with each iteration. Hence, we can show that the right-hand side of the above equation is less than or equal to $\gamma D(s)$, such that

$$D(s) \leq \gamma D(s), \quad (12)$$

Since $\gamma < 1$, the only way for this inequality to hold is when $D(s) = 0$, that is, $V_1^*(s) = V_2^*(s)$. Thus, we have proven the uniqueness of the optimal value function in MDPs. ■

In the Reinforcement Learning (RL) paradigm, the learning agent estimates an action-value function (alternatively, Q-value function) to determine the optimal action in each state. We define the action-value function such that

$$Q^\pi(s, a) = \mathbb{E}^\pi \left[\sum_{t=1}^T \gamma^{t-1} R(t|S_t = s, A_t = a) \right], \quad (13)$$

where $Q^\pi(s, a)$ is the mathematical expectation of the long-term return that is generated by a strategy. According to the goal of MDP, all actions selected by the optimal strategy will result in the maximum return. Hence, we can describe the optimum action-value function and the optimum policy by the following equations, respectively.

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a), \quad (14)$$

$$\pi^*(s) = \arg \max_{\pi} Q^\pi(s, a), \quad (15)$$

This means that for all actions selected by the optimal strategy, that is, $A(t) = \pi^*(s)$, will maximize the action-value function $Q^\pi(s, a)$ in Eq. (13), and satisfy $Q^*(s, a) \geq Q^\pi(s, a), \forall s \in S_t, a \in A_t$.

B. DQN-Based Resource-Efficient Defense Strategy

Although the optimal strategy can be acquired by solving the MDP via dynamic programming or exhaustive search, it is impossible to trace state transition probabilities mathematically because of dynamic defensive actions and complex state space. Recently, RL has been proposed as a promising solution that can be used to solve the optimization problem via trial-and-error. For example, Q-learning is a well-known RL algorithm based on temporal difference, which can learn the optimal strategy by interacting with the environment in a certain period of time and continuously maintaining a Q-value table [63]. According to Bellman equation [61], the Q-value can be expressed as

$$Q(S_t, A_t) = (1 - \lambda) Q(S_t, A_t) + \lambda [R_t + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1})], \quad (16)$$

where λ denotes the learning rate, which affects the learning speed of $Q(S_t, A_t)$.

However, the conventional Q-learning algorithm suffers from several disadvantages, such as state space explosion, overestimation of action values, and long convergence time. It is impractical to adopt Q-learning for this attack-defense scenario with a large state and action space. Fortunately, DRL can effectively capture network dynamics while solving complicated optimization problems with a low time complexity, thereby improving problem-solving efficiency. Hence, DRL is a promising method to address the resource management problem for MTD deployment.

In this regard, we import *Deep Neural Networks* (DNNs) to solve the MDP problem, in which the learning speed will not be significantly affected by the dimension of state and action space. The DQN algorithm transforms the update problem of the Q-value table into a function fitting problem, and the approximate value function is parameterized as

$$Q_\omega(S_t, A_t) \approx Q(S_t, A_t), \quad (17)$$

where ω is the model parameter of the neural network. The objective function value in the network training process gradually approaches the optimal Q value, thereby solving the optimization problem rapidly.

Algorithm 1: DQN-Based MTD Deployment

```

1 Initialize the Q-value network  $Q_\omega(S_t, A_t)$  with  $\omega$ ;
2 Initialize the target network  $Q_{\omega^-}(S_t', A_t')$  with  $\omega^- = \omega$ ;
3 Initialize the experience replay buffer  $B$ ;
4 for  $episode = 1$  to  $K$  do
5   Initialize and acquire the initial state  $S_1$ ;
6   for  $t = 1$  to  $T$  do
7     Select an action  $A_t$  at the state  $S_t$  via the  $\varepsilon$ -greedy
      policy;
8     Compute the immediate reward  $R_t$  via Eq. (4);
9     Obtain the next state  $S_{t+1}$  via Eq. (3);
10    Store transition  $(S_t, A_t, R_t, S_{t+1})$  in  $B$ ;
11    if the capacity of  $B \geq M$  then
12      Randomly select a batch of samples
         $\{(S_i, A_i, R_i, S_{i+1})_{i=1}^M\}$  from  $B$ ;
13      if  $S_i = S_T$  then  $\triangleright$  for terminal time step
14         $y_i = R_i$ ;
15      else  $\triangleright$  for non-terminal time step
16         $y_i = R_i + \gamma \max_{a'} Q_{\omega^-}(S_{i+1}, a')$ ;
17      Minimize  $L = \frac{1}{M} \sum_{i=1}^M (y_i - Q_\omega(S_i, A_i))^2$ ;
18      Reset  $\omega^- = \omega$  every  $C$  steps;

```

1) *Algorithm:* Here, we illustrate the MTD deployment process in Algorithm 1. Specifically, at the beginning of the training period, we initialize the Q-value network with the parameter ω in Line 1. The target network with the same parameter and the experience replay buffer B are initialized in Line 2 and Line 3, respectively. Subsequently, Line 4 starts the main loop of the proposed algorithm, and the iteration starts from an initial state S_1 until finishing T time steps, which is called an episode (Lines 4-6).

In each episode, the agent utilizes ε -greedy policy for selecting an action A_t in Line 7. This implies that it can derive the optimal strategies and avoid staying in local optimization. The algorithm exploits with probability $0 \leq \varepsilon \leq 1$ to take a random action or chooses the optimal strategy with probability $1 - \varepsilon$. It is worth noting that ε initially remains high and gradually decreases after having sufficient experience. Then, Line 8 obtains the immediate reward R_t according to Eq. (4) and Line 9 receives the next state S_{t+1} through the state transition. Since Q-network follows an experience replay approach, the transition (S_t, A_t, R_t, S_{t+1}) is stored in the replay buffer B in Line 10.

In order to ensure stability, the agent will learn from experience and randomly select a batch of M samples from the replay buffer at each time step in Line 12. Then, Line 13 determines if any of the selected samples reached the termination status, that is, whether this training episode ends at this time step. If so, the target value is expressed as $y_i = R_i$, otherwise, it will be represented as $y_i = R_i + \gamma \max_{a'} Q_{\omega^-}(S_{i+1}, a')$, where γ is a discount factor and a' is an arbitrary action. After that, Line 17 minimizes the loss function L at each time step, which is defined as $L = \frac{1}{M} \sum_{i=1}^M (y_i - Q_\omega(S_i, a_i))^2$. Finally, Line 18 updates the parameter of the target network every C steps.

2) *Illustration:* Consider six services with initial state $S_1 = \{s_1(1), s_2(1), s_3(1), s_4(1), s_5(1), s_6(1)\}$ in an EC environment, where $s_1(1)$ is the status of the first service that includes a four-tuple and can be obtained by using Eq. (1). Among these services, two of them (e.g., Service 2 and Service 5) are being under LDDoS attacks, resulting in abnormal service state $(s_2(1), s_5(1))$. If we exploit existing methods such as [32], [52], the network configurations of victim services are continuously changeable according to specific rules or algorithms, which cannot guarantee long-term effectiveness against powerful attackers when they are aware of the adaptation patterns.

Diversely, the proposed DQN-based strategy finds the optimal defensive action via trial-and-error. It may select $A_1 = (a_{1,2}^{hop}, a_{1,5}^{hop})$ at the first time step and obtain the reward R_1 and state S_2 using Eqs. (3) and (4). As the learning phase progresses, it learns experience from the stored buffer, and minimizes the loss function by using Eqs. (5)–(17) and immediate reward at each time step. When attackers have sufficient knowledge and keep waging attacks on these two victim services until T time step, this strategy adjusts defensive actions to $A_T = (a_{T,2}^{sca}, a_{T,5}^{sca})$ according to current Q-value network $Q_\omega(S_T, A_T)$ for reaching the best trade-off between effectiveness and resource consumption. That is, it scales up the victim containers within resource constraints to support more connections and tolerate malicious requests without compromising service quality. Therefore, we can obtain the optimal suggestion for defense deployment according to the DQN algorithm, which allows for improved robustness and resilience against LDDoS attacks by determining effective MTD mechanisms to prevent attack requests or tolerate malicious traffic with lower overhead.

Although this algorithm does not conduct attack detection, changes in service status will become an important indicator of assessing system security and rewards brought by actions. Furthermore, the proposed method is not triggered by specific events, but relies on time steps for decision-making. Thus, a smaller time interval will help timely update and optimize strategies. In practice, defenders may be required to continuously perform or change defensive actions during intense attacks, or they may remain inactive for several time steps to reduce resource usage in the event of low threat.

VI. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our LDDoS mitigation mechanism through extensive simulations and experiments. First, we introduce the values of main parameters used in our simulation and discuss the results under different attack scenarios. And then, we study the effectiveness and resource consumption of the proposed MTD-based mitigation mechanisms and compare them with other practical strategies on the testbed under LDDoS attacks. Finally, we compare our approach with the SOTA methods in real-world experiments.

A. Simulation Setup

First of all, we consider an edge cloud with multiple container-based Web services deployed in our simulation.

TABLE III
KEY PARAMETERS USED IN THE SIMULATION

Parameters	Value or Range
Time slot	5s
Minimum deployment resource $\text{mine}_n(t)$	0.1 CPU & 0.5GB RAM
Capacity of the edge cloud	10 CPU & 50GB RAM
Number of services deployed N	[0, 10]
Connections of a minimum service	[0, 256]
Legitimate users' utilization rate	[50%, 75%]
Range of port numbers $p_n(t)$	[30000, 32767]
Attackers' reconnaissance time	10s
Connections from every attacker	256
Coefficient α , β , ζ , and η	8, 1, 0.02, 0.05

Herein, we build the custom EC environment to train RL agents based on OpenAI Gym [64], which is a widely used *Python*-based toolkit for connecting agents to RL problems. We utilize the official Gym API³ to model environments as a *Python env* class, which enables us to create the EC environment and Web service instances very easily.

In detail, we set the maximum capacity of services with corresponding resource consumption on the edge cloud, and the maximum number of connections for running a Web service is also presented. We have specified the legitimate access of normal users and the available range of the port hopping mechanism in the *env* class. Besides, we simulate LDDoS attackers to send malicious traffic from a certain number of connections after preseted reconnaissance time in the environment. It is worth noting that in order to analyze the performance of the proposed method in different attack scenarios, we have adopted different action configurations for the potential attack targets and attackers' characteristics.

Finally, we execute the proposed MTD mechanisms and DQN algorithm in the simulated EC environment. In each episode, the intelligent agent decides current defensive actions and port configurations, and then distributes these decisions to all services. Table III summarizes the key parameters in our simulation-based experiments, and the main hyperparameters of DNNs are shown in Table IV. Note that the appropriate coefficients in the reward function are determined based on our understanding of the task and by reference to Reference [25], to guide the agent's learning and improve its performance. However, there may be other factors at play, such as the initial state, action selection policies, and environmental changes. Therefore, we discuss the limitations in Section VII.

DRL models often require an adequate number of training iterations to converge to an optimal solution, therefore, it is significant to ensure that it has sufficient time to converge and stabilize its internal representations and weights, leading to improved performance. Nevertheless, excessive episodes may also result in high computational demands. To strike a balance between the training time required and the available computational resources, we performed 10,000 episodes of training and ran a test at every 500 steps, which allows for substantial learning while being mindful of practical considerations. In each test, the summation of all rewards in each episode without a discount factor is viewed as a return.

TABLE IV
HYPERPARAMETER SETTINGS FOR THE DQN ALGORITHM

Parameter	Value or Range
Maximum episodes K	1×10^4
Discount factor γ	0.98
Time steps T per episode	50
Q-network hidden layers	[1024, 128]
Learning rate (Adam)	1×10^{-3}
Decay rate of learning rate	0.2
Decay period of learning rate	4×10^4
Activation function	ReLU
Replay buffer size B	1×10^5
Batch size M	64
Epsilon ε	[0.01, 1]
Epsilon decay rate	1×10^{-5}
Update period of target network C	500

In each test, the current policy was evaluated with the average return during 10 episodes for reducing the uncertainty of the environment. Specifically, we repeated the procedure for ten training runs with random seeds from 1 to 10 to obtain the results, and analyzed the performance of the proposed MTD-based mitigation under four different scenarios according to the threat model in Section III.

In detail, we define these attack scenarios as follows.

- *Attackers with a single and constant (S & C) target.* In this scenario, the LDDoS attackers will select a target initially and continuously attack it until the horizon.
- *Attackers with a single and variable (S & V) target.* The adversaries are able to dynamically select targets based on their knowledge, and they can choose to attack current services or adjust their targets at each step.
- *Attackers with multiple and constant (M & C) targets.* The attackers in this scenario will spread their malicious requests equally across multiple services.
- *Attackers with multiple and variable (S & V) targets.* The attackers will allocate malicious requests to multiple services according to different weights, which can be changeable as needed over time.

B. The Impact of Attack Strength

The effectiveness of the proposed method is evaluated in terms of the average return during the training phase. The performance under different attack strengths is presented in Fig. 3, where the x -axis represents the number of episodes, and the y -axis denotes the average return obtained from the test under the current episode.

Regarding Fig. 3, the average return gradually increases for all the attack scenarios, from a negative value to the highest return of 50 when defending against 10 attackers under M & C and M & V scenarios. The increase implies the effective training of the intelligent agent under different scenarios, and especially, the average return value of 50 indicates that all services are in a normal state with less resource overhead and proves the resource efficiency of the proposed method against low-intensity attacks. Although more attackers pose a more significant challenge to defense methods, our solution still exhibits positive effectiveness (e.g., the average return values

³<https://github.com/openai/gym>

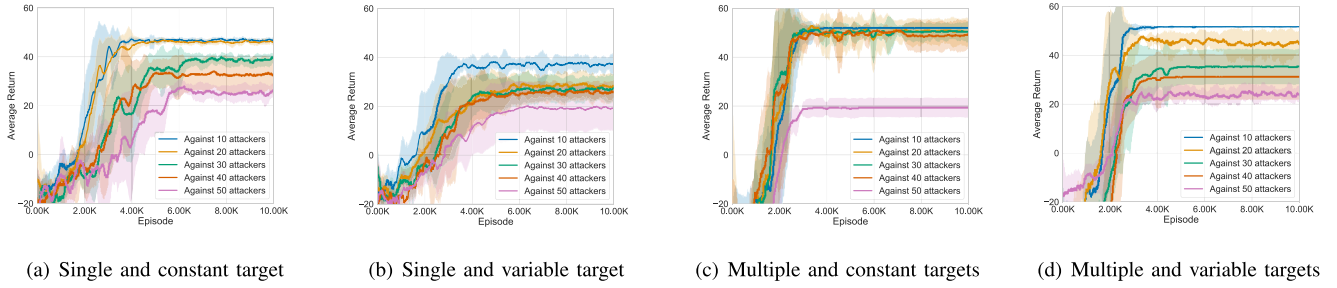


Fig. 3. The average return of the proposed method under different LDDoS attack scenarios. (a) Impact of different numbers of static attackers to wage the same attack target, (b) Impact of different numbers of dynamic attackers to compromise a single attack target, (c) Impact of different numbers of static attackers to simultaneously wage multiple attack targets, (d) Impact of different numbers of dynamic attackers to deliver attacks on multiple targets.

are more than 19 eventually when defeating 50 attackers) due to the guidance from the DQN-based strategy.

Across all the cases in Fig. 3, we can see that the proposed method incurs the lowest effectiveness against 50 attackers, achieving the approximate return value of 24, 19, 20, and 22, respectively. The underlying reason is that when there are 50 attackers, all available connections that the services can handle are occupied according to the simulation setup, resulting in a serious shortage of computational resources for defense. As a result, the intelligent agent can only make decisions in a limited action space and the strategy derived from the DQN algorithm will tend to drive all containers to change their port numbers, leading to a slight decrease in return. Given the resource limitations and similar effectiveness of implementing port hopping, we did not further discuss the returns in scenarios with a higher number of attackers. Moreover, we assume that all the legitimate users request services at a stable rate in our simulation, however, there may exist a sudden surge of requests from a part of newcomers in real scenarios. Therefore, we discuss the limitations in Section VII.

Besides, the maximum return under single-target attack scenarios is around 45 whereas that in multi-target attack scenarios can reach up to 50. This may be attributed to two reasons, the first is that the strategy space of the proposed method is limited when dealing with single-target attacks, because it is necessary to avoid damaging the security status of other normal services. The second is that when the system recovers from multi-target attacks, it will incur a higher return based on the reward function. Another interesting observation is that when defending against attackers with multiple and constant targets, the effectiveness is almost the same, ranging from 45 to 50 if there are no more than 40 attackers, as illustrated in Fig. 3(c). This can be attributed to the attack characteristic in which attackers evenly distribute malicious requests, MTD techniques can fully utilize existing sufficient resources to effectively resist attacks and improve security. Note that the performance in most variable scenarios is lower than that in constant scenarios. This phenomenon indicates that attackers will be more threatening when they can dynamically adjust their attack targets, and also proves the necessity of designing MTD mechanisms to deal with them.

There is an interesting observation that in scenarios with a constant attack target and a relatively small attacker population, the proposed methodology has demonstrated remarkably similar

reward outcomes (can be seen in Fig. 3 (a) and (c)). For instance, the proposed method achieved a return value of around 50 when defending against 10, 20, and 30 attackers in the M & V scenario. That is, our approach selects similar defensive actions under relatively low attack intensity, resulting in comparable defensive effectiveness. This phenomenon can be attributed to the fact that in scenarios with a constant attack target, the proposed method, when facing relatively low-intensity attacks, tends to prioritize the service scaling and replication mechanisms through the DQN algorithm to minimize unnecessary time overhead. However, as the attack intensity increases and available resources gradually diminish, the method shifts its focus to the port hopping mechanism to ensure security. However, this observation has not been observed in the multi-target scenarios, particularly in the variable multi-target setting. In this case (can be seen in Fig. 3 (d)), the defensive reward continually decreases as the attack intensity increases with the number of attackers. The underlying reasons for this are twofold. Firstly, the protected targets need to be dynamically adjusted due to changes in the attack strategy. Secondly, the resource scheduling for the multi-target scenario is more complex, making it infeasible to achieve high defensive rewards through similar defensive actions alone.

C. The Impact of MTD mechanisms

To analyze the impact of different MTD mechanisms on the effectiveness and to demonstrate the necessity of the three mechanisms designed in this work, we individually evaluate the performance of the three MTD mechanisms as well as their combined application against the aforementioned four kinds of attack scenarios in this section. Additionally, in low-intensity attack scenarios where resources are relatively abundant, the differences in defensive reward resulting from varying defensive actions cannot be fully manifested. Therefore, to validate the importance of appropriate defensive action orchestration with a high level of intensity, we compare the effect of service scaling, service replicas, port hopping, and their combined deployment in terms of the average return against 50 LDDoS attackers.

As we can see from Fig. 4, the combined MTD mechanisms have a significant advantage over any of a single mechanism, achieving the average return value of around 28, 20, 20, and 26 on all attack scenarios. Especially, when dealing with the

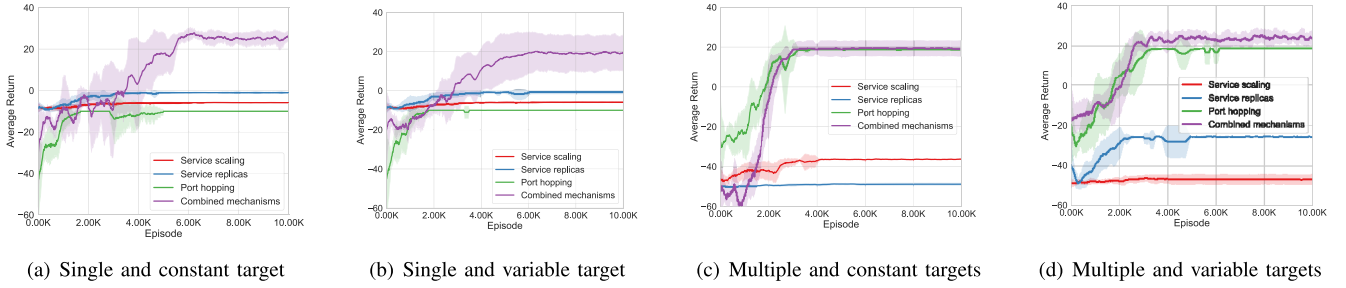


Fig. 4. The comparison of the proposed MTD mechanisms under different LDDoS attack scenarios. (a) Impact of different defense mechanisms against constant single-target attacks, (b) Impact of different MTD techniques to defeat variable single-target attacks, (c) Impact of different MTD mechanisms to defend against constant multi-target attacks, (d) Impact of different defensive methods to mitigate variable multi-target attacks.

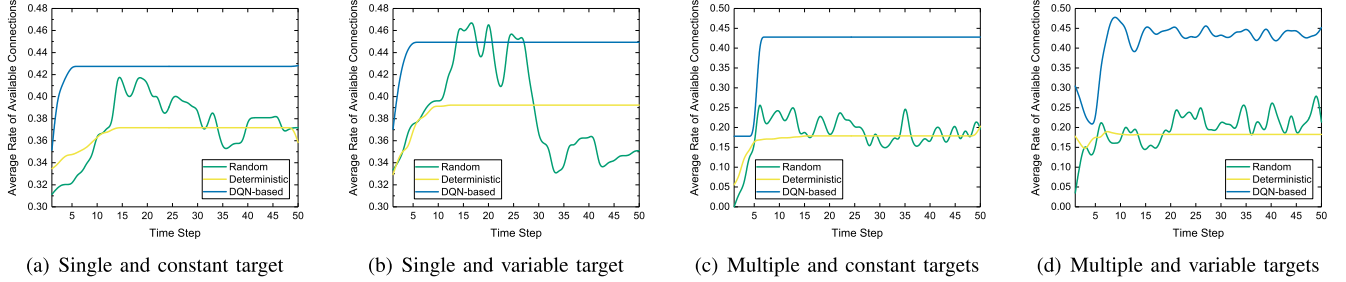


Fig. 5. The comparison of the proposed DQN-based MTD deployment strategy with two baseline strategies under different LDDoS attack scenarios. (a) Comparison of different defensive strategies against constant single-target attacks, (b) Impact of different MTD strategies to defeat variable single-target attacks, (c) Impact of different deployment strategies to defend against constant multi-target attacks, (d) Effectiveness of different strategies to mitigate variable multi-target attacks.

constant single-target attack, there is a significant increase in the average return (at most from -10 to 28) after switching the port hopping to the combined mechanisms. The underlying reason is that independently deploying an MTD mechanism is ineffective in mitigating LDDoS attacks, whereas the combination of multiple MTD mechanisms can seamlessly work together to strengthen the defense, thereby resulting in a higher return value. Moreover, it is worth noting that both service scaling and service replicas will bring negative returns across all scenarios, achieving the return value of at least -5 , -5 , -50 , and -47 , when defeating powerful LDDoS attacks. This result discloses that the unreasonable MTD deployment may not increase the effectiveness against attacks, but will cause unnecessary waste of computing resources.

With more details, we further observe that the port hopping is similarly effective to the combined mechanisms, achieving an average return value of around 20 under the multi-target attack scenarios. Due to the settings of the simulation environment, all available connections that the services can handle are occupied, leaving insufficient idle computational resources for scaling up or creating replicas when there are 50 attackers. Therefore, we cannot defend through the scheduling of physical resources and must instead choose port hopping to block LDDoS attack traffic. Although it may affect the experience of current legitimate users, it effectively maintains service availability under high-intensity attacks. Even in the presence of a larger number of attackers, as defense remains restricted to port hopping, the defensive rewards will be similar to the scenario with 50 attackers. Since there is no significant difference in returns, we did not further discuss scenarios with a higher number of attackers in our study. Last but not least, according to the results among all scenarios, we

can conclude that the proposed solution can optimize the scheduling of different MTD mechanisms, thereby improving resource efficiency significantly.

D. The Impact of Deployment Strategies

To further examine the effectiveness and overhead of our approach, particularly the impact of different decision-making methods, we consider the aforementioned attack scenarios against LDDoS adversaries, and compare the proposed DQN-based policy with two baseline strategies as follows.

- *Random strategy.* The defender randomly performs an action from the proposed MTD mechanisms on a random service at each time step.
- *Deterministic strategy.* The defender selects a deterministic action on the victim services according to the current system status and expert knowledge.

Similar to Section VI-C, we also set up an LDDoS attack scenario with 50 attackers to investigate the Web performance of the aforementioned strategies under high-intensity attack conditions in this section. We first measure the number of connections induced by the different defense strategies, thereby evaluating the Web service quality. Subsequently, we compare the number of Web services protected by the proposed policy and other baseline strategies across all edge environments. Finally, we analyze the defensive resource utilization required for deploying these strategies under varying attack scenarios.

1) *QoS Evaluation:* In order to verify the capability of the proposed approach to maintain QoS while guaranteeing efficient performance, we evaluate the average rate of available connections for all the Web services deployed on the edge cloud. Fig. 5 depicts the QoS performance of different

TABLE V
RESOURCE CONSUMPTION OF THE DIFFERENT DEPLOYMENT STRATEGIES WHEN DEFENDING AGAINST 50 LDDoS ATTACKERS

Deployment strategy	S & C scenario		S & V scenario		M & C scenario		M & V scenario	
	Avg.	StdDev.	Avg.	StdDev.	Avg.	StdDev.	Avg.	StdDev.
Random Strategy	31.09	16.05	28.46	16.69	42.79	14.34	40.85	15.05
Deterministic Strategy	42.49	11.61	40.07	12.23	46.17	9.60	48.18	7.96
DQN-based Strategy	45.24	8.07	44.32	8.04	49.80	2.44	49.60	3.44

defensive strategies against four kinds of LDDoS attacks, where three broken lines represent the random strategy, deterministic strategy, and DQN-based strategy.

As we can see, the deterministic strategy exhibits a gradual upward trend and finally achieves an available connection rate of 0.372, 0.392, 0.177, and 0.184 across four kinds of attack scenarios. Similarly, the proposed DQN-based strategy also reaches convergence as the time step increases, achieving the available connection rate of 0.428, 0.45, 0.428, and 0.442, respectively. This observation indicates that these strategies will ensure sufficient resources for providing network services to legitimate users. Meanwhile, the DQN-based strategy achieves a higher available connection rate (at most a 26.95% increase under the variable multi-target scenario) when compared with the deterministic strategy. This can be attributed to the success of the DRL method in obtaining a better strategy than that of acquiring expert knowledge, leading to higher QoS within limited resources.

Nevertheless, the QoS of the random policy exceeds the DQN-based policy in several time steps under the variable single-target scenario, achieving at highest connection rate of 0.467, as illustrated in Fig. 5(b). This is because the randomly selected action has a certain probability to significantly increase QoS, but it may bring unnecessary overhead and incur lower effectiveness. Furthermore, there is a dramatic fluctuation in the QoS of the random strategy across all the scenarios. Especially, the connection rate of this strategy fluctuates between 0.334 and 0.467 under the S & V scenario. The underlying reason is that unreasonable defense deployment driven by the random policy may affect the normal operation of the Web service itself, causing instability in the QoS and affecting the user experience. Therefore, we can conclude that the proposed DQN-based strategy can help the defender maintain a high level of QoS and provide users with stable Web services even under high-intensity LDDoS attacks.

2) *Security Evaluation*: To intuitively exhibit the performance of our approach against LDDoS attacks, we first measure the average number of surviving services during 50 time steps. Then, the results under different attack scenarios are compared with other strategies, as illustrated in Fig. 6.

As we can see, the DQN-based policy can protect more services from attacks than the other two strategies in all scenarios, achieving the average survivability rate of 0.98, 0.97, 0.97, and 0.95, respectively. Especially, the average number of available services with our method is almost 1.5 times that under the random policy and the deterministic strategy, when defending against attackers with multiple and variable targets. And beyond that, the other two strategies can prevent at most 63.8% of Web services from being attacked

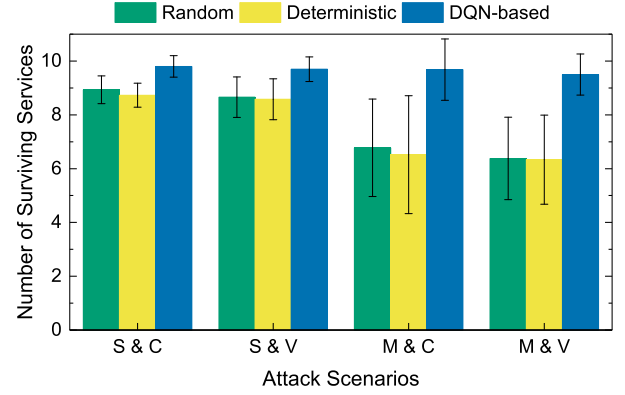


Fig. 6. The comparison of different deployment strategies for implementing the MTD mechanisms against powerful LDDoS attacks.

under this scenario. Due to the guidance of the DQN-based strategy, our approach is capable of keeping more than 8 Web services running at all times. The results disclose that, regardless of the behavior and preference of the attackers, the resistance of the proposed method is significantly higher than using other defensive strategies.

Another interesting observation is that the average performance of the random policy is slightly higher (at most a 4% increase under the M & C scenario) than that of the deterministic strategy. The underlying reason may be that the random strategy does not consider the negative effects it may have on QoS (see Fig. 5) and happens to improve security due to some extreme actions. In addition, the lower effectiveness of the deterministic strategy suggests that consistently following certain patterns, rather than dynamically selecting the optimal defensive strategy, would negatively impact security.

3) *Overhead Evaluation*: To provide some additional insights on the defense resource consumption against attackers with different attack strengths, we report the resource usage for the defense deployment according to these three strategies, which is presented in Table V. Note that the resource consumption is quantitatively measured based on the resource usage of the minimum service in Table III, that is, 1 means that this strategy will use 0.1-core CPU and 0.5 GB RAM.

We observe that when compared with the multi-target scenarios, the defense resource consumption of all three strategies is lower in the S&C and S&V scenarios. For example, the random strategy incurs an average resource consumption of 28.46 under the S & V scenario while 42.79 units of defensive resources need to be involved in the M & C scenario. The reason for this observation may be that more resources need to be mobilized to participate in defensive actions when the targets are relatively scattered. Meanwhile, there is no

significant difference (at most 2.63 units of resources incurred by the random strategy) in resource usage between the constant scenario and variable scenario. This result may indicate that the resource consumption needed for defense deployment is not so much determined by the attacker's behavior patterns in selecting attack targets. Instead, it is largely influenced by the intensity and scale of the attack. As another example, attackers under the variable scenario may choose the same or similar target as in a constant scenario over several time steps, which will also lead to similar defensive actions and resource utilization.

It is also worth noting that the random strategy consumes 31.09, 28.46, 42.79, and 40.85 units of resources on average under all four kinds of scenarios, representing the lowest resource utilization among all the strategies under consideration. However, it is difficult to make this strategy a high resource efficiency one due to the unstable QoS and lower survivability brought by it. Although the resource usage of the DQN-based strategy proposed in this paper is only slightly higher than that of the deterministic strategy, it can be considered an acceptable overhead because of its significant improvement in QoS (at most a 26.95% increase) and security (at most a 31.7% increase) evaluated in Sections VI-D1 and VI-D2. There is another observation that the DQN-based policy consumes 45.24, 44.32, 49.8, and 49.6 units of defensive resources across all attack scenarios. The resource usage of our method is very close to 50, which is exactly the maximum amount of resources we can afford in our simulations. This result proves that the proposed approach can maximize the utilization of limited resources and achieve a perfect balance between defense effectiveness and resource utilization when defending against high-intensity attacks.

E. Comparison With Previous Works

We conduct experiments in a real-world testbed and compare our approach with the SOTA methods as follows.

- *OpenMTD* [65]. It employs a hybrid approach that integrates network address shuffling and port hopping techniques to resist attacks. The high-frequency and low-frequency mutation cycles are set at 30 and 120 seconds, and the hopping period is 20 seconds.
- *FastMove* [66]. It protects Web servers from DDoS attacks using a network address change mechanism through multiple proxies, in which we set 10 replicas for each Web server and a time interval of 10 seconds in our experiments.
- *CMDP-MOS* [50]. It combines admission control and service migration to mitigate DDoS attacks. Also, it adopts the constrained Markov decision processes (CMDP) to model the optimization problem and presents an MTD optimal strategy (MOS) algorithm to guide defense deployment.
- *ID-HAM* [67]. It integrates host address mutation (HAM) with deep reinforcement learning to slow down network reconnaissance intelligently. In detail, it transforms the real IP addresses of hosts into virtual ones by OpenFlow

switch and adopts an advantage actor-critic algorithm for HAM.

In our experiments, the testbed is composed of a Dell PowerEdge R630 server and a Dell PowerEdge R430 server. Each Dell PowerEdge R630 has 24-core CPU, 64 GB of RAM, and 4 TB hard disk storage, and Dell PowerEdge R430 has 4-core CPU, 16 GB RAM, and 120 GB disk storage. The SDN controller is deployed using Ryu⁴ v4.3.4 on the Dell PowerEdge R430 server, in which we also deploy several MTD mechanisms and the intelligent agent as modules. For the edge cloud deployment, we utilize OpenStack Queens Devstack⁵ in another server, and we deploy containerized services with some built-in functions, including scaling up/down, creating replicas, and container migration. In addition, the bandwidth of each link is configured as 1 Gbps.

In summary, we include a controller and four edge clouds, each with a capacity of 16GB RAM, in the testbed. We deploy a free and open Web service namely bWAPP⁶ in each edge by pulling the image from DockerHub. We use the Apache HTTP server benchmarking tool⁷ to simulate normal user access requests, where the number of concurrent connections varies to represent different system loads. To evaluate the effectiveness of these methods against high-intensity and large-scale attacks, we launch LDDoS attacks on all running services by using the SlowHTTPTest tool. Specifically, we utilize a 10-second time interval and generate a total of 100 connections per second for each edge cloud, and track the attack target if the port number has been altered. In this scenario, without the implementation of any defense mechanisms, all Web services would become inaccessible due to being overwhelmed by a large volume of malicious connections.

Furthermore, to assess the performance of various methods under different resource constraint conditions, we configure varying numbers of concurrent user connections μ . For instance, when $\mu = 200$, the protected system has relatively abundant available resources. Conversely, when the concurrency level reaches 800, the system must allocate a substantial amount of resources to handle user requests, leaving only a diminished portion for defense deployment.

1) *Response Time*: To analyze the effectiveness of different defensive schemes, we evaluate the average response time of each method on the server side by simulating a total of 10,000 normal requests and LDDoS attack requests lasting for 300 seconds. The results under different system loads are presented in Fig. 7, where the x-axis represents the percentage of requests served, and the y-axis denotes the response time obtained from the test within these parts of requests.

We can make the first observation that the response time of all methods gradually increases as the number of concurrent connections grows. This increase implies the impact of system loads on the response time is significant when dealing with attacks of equal intensity. We can also observe that the proposed method consistently exhibits the lowest response

⁴<https://ryu-sdn.org>

⁵<https://opendev.org/openstack/devstack>

⁶<https://hub.docker.com/r/raesene/bwapp>

⁷<https://httpd.apache.org/docs/current/programs/ab.html>

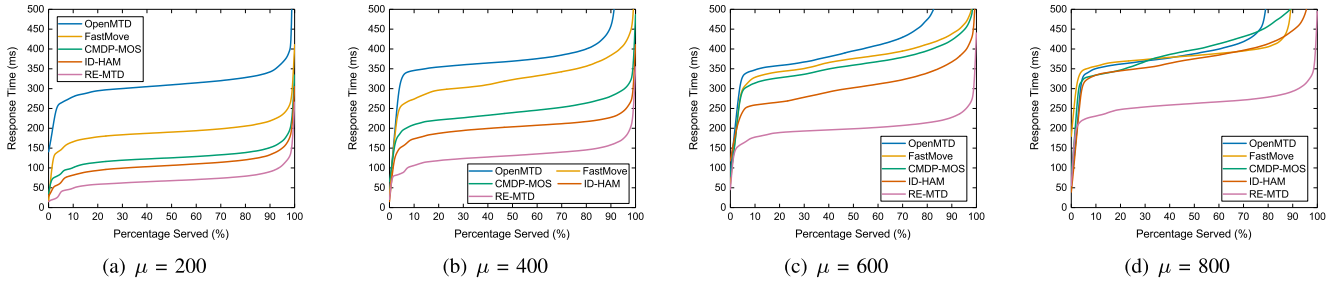


Fig. 7. The request time of different schemes when defeating LDDoS attacks under different system loads, where μ represents the number of concurrent connections. (a) Comparison of different defensive methods when requests come from 200 concurrent connections, (b) Impact of different approaches to defeat attacks within 400 concurrent connections, (c) Impact of different solutions to defend against attacks under 600 concurrent connections, (d) Response efficiency of different schemes to mitigate attacks when there are 800 concurrent connections.

time across various system workloads. Specifically, under lower workloads such as the scenario with 200 concurrent connections, the RE-MTD approach achieves a response time of less than 100 ms for 93% of the requests. In contrast, the FastMove, CMDP-MOS, and ID-HAM methods only respond to requests within this timeframe for 2%, 11%, and 32% of the requests, respectively. Notably, the OpenMTD method fails to achieve this for any request. Moreover, under the condition of 800 concurrent connections, the RE-MTD method is able to effectively handle requests, with only 6% of requests requiring 300ms or more response time. In contrast, the other methods exhibit longer response times, particularly with FastMove, where 97% of requests fail to complete within 300ms. This substantiates that the proposed approach can effectively mitigate attacks, which prevents the Web service from being overwhelmed by a large number of connections initiated by attackers, even in the presence of limited available resources. As a result, it is capable of effectively safeguarding legitimate user access and can operate efficiently under varying resource constraint conditions.

Furthermore, there is another interesting observation that the advantage of RE-MTD over other methods also gradually expands as the number of concurrent connections increases. For instance, when handling 80% of the requests, compared to the second-best method, RE-MTD reduces the response time by approximately 40 ms, 70 ms, 120 ms, and 130 ms for concurrent connection counts of 200, 400, 600, and 800, respectively. The underlying reason may be that the proposed method in this study incorporates Web service quality as a crucial component of the reward function in the MDP modeling. Additionally, the MTD mechanisms devised in this study, such as service scaling and service replicas, do not differentiate between attack-induced states and high-load states. Consequently, while other MTD methods may be limited to addressing attacks, RE-MTD adopts a broader global perspective. Even under high workload conditions where there are insufficient defensive resources, it leverages the DQN algorithm to output the optimal strategy, thereby maximizing service performance within the current resource constraints. As such, RE-MTD not only effectively handles the malicious connections generated by attacks, but also simultaneously optimizes the load balancing and system performance for the substantial volume of legitimate user requests.

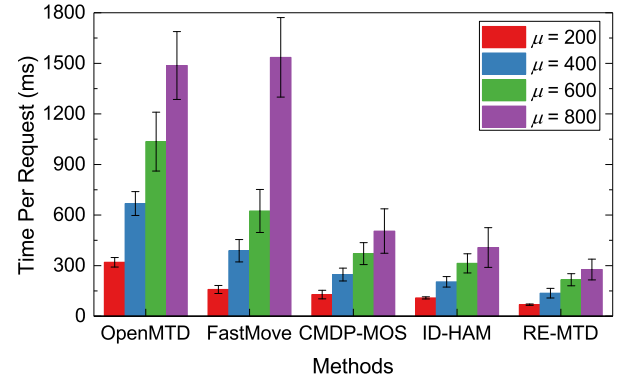


Fig. 8. The comparison of time per request experienced by users when equipped with different defensive schemes.

2) *Quality of Experience*: In order to assess the user experience from the user's perspective during webpage access, we measured both the average response time per request and the webpage loading duration in our experiments. As depicted in Fig. 8, we can observe that as the system workload increases, the speed at which the Web service responds to user requests slows down, consistent with the results obtained from our server-side testing (see Fig. 7). This is attributed to the increased processing pressure on the server due to a higher number of concurrent connections, resulting in longer response times for each request.

Nevertheless, we note that RE-MTD still exhibits the lowest average response time, reaching 68.36 ms, 136.50 ms, 215.98 ms, and 276.66 ms for 200, 400, 600, and 800 concurrent connections, respectively. Particularly in the high workload scenario ($\mu = 800$), RE-MTD outperforms alternative approaches such as OpenMTD and FastMove. By reducing the response time by over 1 second, RE-MTD is able to ensure a favorable user experience even when there are insufficient defensive resources available. This phenomenon can be attributed to the fact that OpenMTD and FastMove methods lack defense designs specifically targeting low-rate attack scenarios and the fixed frequency of adaptations is not efficient in defending against dynamically evolving attackers. As a result, malicious connections impede the response of legitimate requests.

Fig. 9 illustrates the webpage loading times when deploying different defense methods, with each subplot representing a

TABLE VI
MEMORY USAGE RATIOS OF DIFFERENT SCHEMES WHEN DEFENDING AGAINST LDDoS ATTACKS UNDER DIFFERENT WEB SERVER LOADS

Scheme	$\mu = 200$			$\mu = 400$			$\mu = 600$			$\mu = 800$		
	Min.	Max.	Avg.	Min.	Max.	Avg.	Min.	Max.	Avg.	Min.	Max.	Avg.
OpenMTD [65]	20.66	48.83	27.43	47.43	93.34	62.47	42.71	97.27	80.32	70.44	96.76	84.06
FastMove [66]	42.01	63.64	49.79	50.50	76.69	64.08	62.02	95.99	79.33	77.78	98.37	92.62
CMDP-MOS [50]	12.28	38.97	19.88	46.76	70.86	57.14	49.81	95.53	73.71	78.12	98.88	87.97
ID-HAM [67]	7.67	22.69	11.57	42.12	48.76	44.62	46.85	51.10	49.01	63.65	97.38	76.19
RE-MTD (Proposed)	8.25	31.84	14.01	36.76	62.59	47.16	50.74	66.33	54.98	75.34	88.17	78.63

μ : Number of Concurrent Connections; Min.: Minimum Value; Max.: Maximum Value; Avg.: Average Value.

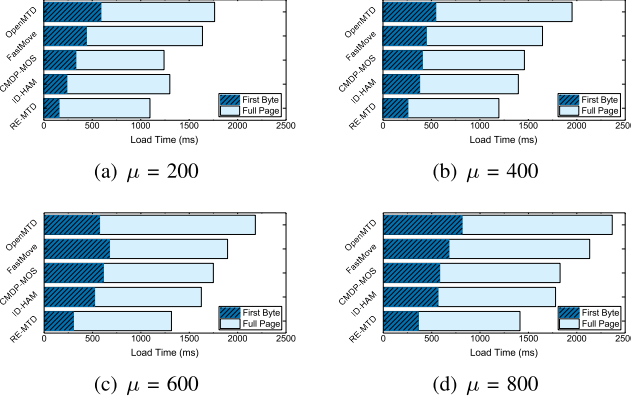


Fig. 9. The average load time of Web services when deploying different solutions under different service loads. (a) Comparison of different methods when requests come from 200 concurrent connections, (b) Impact of different schemes on the load time within 400 concurrent connections, (c) Impact of different approaches on dealing with 600 concurrent connections, (d) Web performance of different solutions when there are 800 concurrent connections.

different system workload level. The dark blue shaded area represents the loading time of the first byte, while the light blue bars indicate the loading time of the full webpage. As the RE-MTD method simultaneously considers both defense effectiveness and service quality in its defensive decision-making, we also observe that it exhibits significant advantages over other approaches in terms of the loading times of both the first byte and the complete webpage across various workload conditions. This demonstrates the excellent scalability of the proposed method under different scenarios. Particularly, when $\mu = 800$, RE-MTD completes the loading of the full webpage within 1413 ms. In contrast, ID-HAM and CMDP-MOS require approximately 1800 milliseconds, FastMove takes over 2100 milliseconds, and OpenMTD approaches nearly 2500 milliseconds. In terms of the loading time of the first byte, the proposed method requires only 362ms, while FastMove and OpenMTD take 679 ms and 815 ms, respectively, which is nearly 1.9 times and 2.3 times that of RE-MTD. The underlying reason could be that ID-HAM and CMDP-MOS are more inclined towards eliminating and blocking attacks, thereby neglecting performance optimization in resource-constrained scenarios. On the other hand, OpenMTD and FastMove employ fixed time intervals for network address shuffling or port hopping, which inadvertently provide opportunities for strategically-minded attackers and result in longer load time because of malicious connections.

3) *Resource Consumption*: Due to the limited resources of edge devices, it is essential to develop defense methods that are highly resource-efficient. Therefore, in this section, we evaluate the memory usage of the aforementioned defense methods under different system workloads and present experimental results in Table VI.

As we can see, the memory overhead required for deploying various methods increases noticeably as the number of concurrent connections grows. This substantiates that under a fixed total resource capacity, a higher volume of user requests will result in more constrained resources available for defense. FastMove exhibits the highest or near-highest memory usage rate in all scenarios, reaching 49.79%, 64.08%, 79.33%, and 92.62%, respectively. Particularly in the high workload scenario ($\mu = 800$), FastMove even reaches a maximum memory usage ratio of 98.37%, imposing a significant burden on the normal operation of the system. This can be attributed to FastMove directly creating 10 replicas for each Web service without dynamic adjustments during runtime, thus consuming a substantial amount of system resources.

Furthermore, we can make an observation that CMDP-MOS performs better than OpenMTD at lower workloads, but it consumes more system resources at higher workloads, with an average increase of 3.91% in memory usage ratio at $\mu = 800$. The underlying reason for this could be the different MTD mechanisms employed by the two methods. Specifically, OpenMTD utilizes network address shuffling and port hopping to prevent attackers from probing and targeting real services, while CMDP-MOS employs admission control and container migration for collaborative defense across multiple edge servers. Therefore, at lower workloads, CMDP-MOS filters out malicious connections through its admission control mechanism, while OpenMTD requires resource-intensive address and port adjustments. However, at higher workloads, CMDP-MOS may utilize container migration to transfer the promised service to another server, introducing higher system resource consumption.

There is another important observation that ID-HAM exhibits the lowest resource overhead, with memory usage rates of 11.57%, 44.62%, 49.01%, and 76.19% at $\mu = 200, 400, 600$, and 800, respectively. The proposed method in this paper follows closely, requiring only an additional 2.44%, 2.54%, 5.97%, and 2.44% of memory usage, respectively, without incurring further memory consumption due to changes in resource constraint conditions. Similarly, this can be attributed to the differences in MTD mechanisms. Since

the host address mutation module of ID-HAM is deployed on the SDN controller and switches, there are no additional resource overheads on the Web service side. On the other hand, the container scaling and replica creation mechanisms of RE-MTD are deployed on the service side, inevitably resulting in increased overheads. However, ID-HAM specifically targets the network reconnaissance phase and lacks a defense mechanism designed to counter LDDoS attacks. Its higher request response time and webpage loading time indirectly indicate its disadvantage in terms of defense effectiveness. In contrast, RE-MTD addresses the limitations of existing MTD methods, demonstrating the ability to dynamically maintain service availability even under high workloads and low resource availability conditions. Given the advantages of the proposed method in terms of defense effectiveness and service quality assurance, as well as its efficient deployment under various resource constraint conditions, we conclude that this approach is capable of addressing LDDoS attacks in a highly resource-efficient manner.

VII. LIMITATIONS AND OPEN ISSUES

Despite the very promising results achieved by the proposed MTD-based method, it is clear that such an approach exhibits some limitations. First, as previously discussed, resource competition is actually the most critical part of the confrontation between the attacker and the defense, the proposed method cannot withstand a radical and powerful attacker whose resource ownership is far greater than that of the defender. In this case, the attacker may send malicious requests to all serving containers simultaneously, limiting the dimensions of action space. Note that this is the result of extreme resource inequality between both sides rather than a vulnerability of the approach itself. Therefore, we can solve it via cooperative defense among multiple edges, which can be investigated in the future.

The proposed approach focuses on effective defense against external attacks in the EC environment. Nevertheless, the MTD-based solution may fail when the adversary gains sufficient knowledge of the target system, and especially in the case that he/she is an insider attacker who has access to sensitive resources and privileged system accounts. Thus, it might be very difficult to defeat these attacks because legitimate users and insider attackers cannot be distinguished especially at the beginning of attacks. Further, this attacker can amplify the attack effect by blocking the MTD functionality and hijacking other nearby edges. Fortunately, some solutions have been proposed to deal with the insider threat, such as security information and event management or zero trust policy. In this respect, it is also a challenge to seamlessly integrate MTD with existing defensive methods to maximize resource efficiency.

Another limitation of our approach may be its performance would be affected by the network environment or user behaviors. In this work, we assume that the network environment is relatively stable and legitimate users access Web services in a consistent frequency. However, both increased demand and network upgrades can result in significant changes in network availability and performance. Although this is not caused by

offensive and defensive actions, it may significantly affect the effectiveness of the proposed method in the real world. A more detailed model formulation may be a possible solution, in which we can incorporate random noise into the description of the environment, and utilize data-driven approaches to automatically determine the parameters to fit the practical scenario.

In the end, although our approach is not bulletproof, we believe that it significantly improves the security of the EC environment and ensures the QoS under LDDoS attacks in a resource-efficient manner.

VIII. CONCLUSION AND FUTURE WORK

Owing to the advantages of MTD techniques, existing work has shown that MTD-based solutions can mitigate DDoS attacks in a more proactive and effective way. However, most of the recent studies were not suitable for hard-to-detect LDDoS attacks, which are usually launched by stealthy attackers. In addition, the high resource consumption of current solutions limits their deployment in the EC environment.

To cope with the aforementioned challenges, we have exploited MTD to defeat LDDoS attacks in a proactive manner, by continuously shifting the network properties and dynamically misleading attackers. In detail, we have first designed three lightweight MTD mechanisms via the built-in functionalities of the container-based applications. We have then considered optimizing the MTD deployment by exploiting MDP and a DQN algorithm to achieve resource-efficient defense. The simulations demonstrate the effectiveness of our method and its advantages over other practical strategies in terms of performance and overhead. To the best of our knowledge, this is the first to overcome the challenge of LDDoS attacks through the deployment of MTD. This approach allows one to significantly improve edge security without high resource consumption.

In future work, we plan to investigate a distributed collaborative defense framework among multiple edge clouds to resist cyber attacks. Based on diverse MTD mechanisms and flexible virtualization technologies, edge servers will be able to jointly defend against large-scale DDoS attacks without encountering resource bottlenecks. Another future development is to extend our work to defeat more advanced attackers, who may be capable of developing attack strategies through reinforcement learning. We aim to defeat intelligent LDDoS attackers while optimizing the efficiency and resource consumption of defense.

REFERENCES

- [1] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: State of the art and research challenges," *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 430–447, Apr. 2018.
- [2] P. Kumar and R. Kumar, "Issues and challenges of load balancing techniques in cloud computing: A survey," *ACM Comput. Surveys*, vol. 51, no. 6, pp. 1–35, 2019.
- [3] F. Metzger, T. Hofffeld, A. Bauer, S. Kounev, and P. E. Heegaard, "Modeling of aggregated IoT traffic and its application to an IoT cloud," *Proc. IEEE*, vol. 107, no. 4, pp. 679–694, Apr. 2019.
- [4] Q. Duan, S. Wang, and N. Ansari, "Convergence of networking and cloud/edge computing: Status, challenges, and opportunities," *IEEE Netw.*, vol. 34, no. 6, pp. 148–155, Dec. 2020.

- [5] K. Wang, Y. Wang, Y. Sun, S. Guo, and J. Wu, "Green Industrial Internet of Things architecture: An energy-efficient perspective," *IEEE Commun. Mag.*, vol. 54, no. 12, pp. 48–54, Dec. 2016.
- [6] Y. Wu, "Cloud-edge orchestration for the Internet of Things: Architecture and AI-powered data processing," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12792–12805, Aug. 2021.
- [7] Y. Zhang, X. Lan, J. Ren, and L. Cai, "Efficient computing resource sharing for mobile edge-cloud computing networks," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1227–1240, Jun. 2020.
- [8] C. Xu, X. Ma, R. Shea, H. Wang, and J. Liu, "Enhancing performance and energy efficiency for hybrid workloads in virtualized cloud environment," *IEEE Trans. Cloud Comput.*, vol. 9, no. 1, pp. 168–181, Mar. 2021.
- [9] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Trans. Cloud Comput.*, vol. 7, no. 3, pp. 677–692, Sep. 2019.
- [10] R. Zhou, Z. Li, and C. Wu, "An efficient online placement scheme for cloud container clusters," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1046–1058, May 2019.
- [11] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient live migration of edge services leveraging container layered storage," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 2020–2033, Sep. 2019.
- [12] R. Gupta, I. Gupta, A. K. Singh, D. Saxena, and C.-N. Lee, "An IoT-centric data protection method for preserving security and privacy in cloud," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2445–2454, Jun. 2023.
- [13] R. Gupta, D. Saxena, I. Gupta, and A. K. Singh, "Differential and TriPhase adaptive learning-based privacy-preserving model for medical data in cloud environment," *IEEE Netw. Lett.*, vol. 4, no. 4, pp. 217–221, Dec. 2022.
- [14] J. Ni, X. Lin, and X. S. Shen, "Toward edge-assisted Internet of Things: From security and efficiency perspectives," *IEEE Netw.*, vol. 33, no. 2, pp. 50–57, Apr. 2019.
- [15] I. Stelios, P. Kotzanikolaou, M. Psarakis, C. Alcaraz, and J. Lopez, "A survey of IoT-enabled cyberattacks: Assessing attack paths to critical infrastructures and services," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3453–3495, 4th Quart., 2018.
- [16] M. Du and K. Wang, "An SDN-enabled pseudo-honeypot strategy for distributed denial of service attacks in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 16, no. 1, pp. 648–657, Jan. 2020.
- [17] Q. He et al., "A game-theoretical approach for mitigating edge DDoS attack," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 4, pp. 2333–2348, Aug. 2022.
- [18] R. Gupta, D. Saxena, I. Gupta, A. Makkar, and A. K. Singh, "Quantum machine learning driven malicious user prediction for cloud network communications," *IEEE Netw. Lett.*, vol. 4, no. 4, pp. 174–178, Dec. 2022.
- [19] Z. Li, H. Jin, D. Zou, and B. Yuan, "Exploring new opportunities to defeat low-rate DDoS attack in container-based cloud environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 3, pp. 695–706, Mar. 2020.
- [20] D. Tang, Y. Yan, S. Zhang, J. Chen, and Z. Qin, "Performance and features: Mitigating the low-rate TCP-targeted DoS attack via SDN," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 428–444, Jan. 2022.
- [21] L. Zhou, H. Guo, and G. Deng, "A fog computing based approach to DDoS mitigation in IIoT systems," *Comput. Secur.*, vol. 85, pp. 51–62, Aug. 2019.
- [22] Y. Zhou, G. Cheng, S. Jiang, and M. Dai, "Building an efficient intrusion detection system based on feature selection and ensemble classifier," *Comput. Netw.*, vol. 174, Jun. 2020, Art. no. 107247.
- [23] Z. Liu, H. Jin, Y.-C. Hu, and M. Bailey, "Practical proactive DDoS-attack mitigation via endpoint-driven in-network traffic control," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1948–1961, Aug. 2018.
- [24] Y. Zhou, G. Cheng, and S. Yu, "An SDN-enabled proactive defense framework for DDoS mitigation in IoT networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 5366–5380, 2021.
- [25] T. Zhang et al., "How to mitigate DDoS intelligently in SD-IoV: A moving target defense approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1097–1106, Jan. 2023.
- [26] J.-H. Cho et al., "Toward proactive, adaptive defense: A survey on moving target defense," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 709–745, 1st Quart., 2020.
- [27] J. Tan et al., "A survey: When moving target defense meets game theory," *Comput. Sci. Rev.*, vol. 48, May 2023, Art. no. 100544.
- [28] Y. Zhou, G. Cheng, S. Jiang, Y. Zhao, and Z. Chen, "Cost-effective moving target defense against DDoS attacks using trilateral game and multi-objective Markov decision processes," *Comput. Secur.*, vol. 97, Oct. 2020, Art. no. 101976.
- [29] N. Agrawal and S. Tapaswi, "Defense mechanisms against DDoS attacks in a cloud computing environment: State-of-the-art and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3769–3795, 4th Quart., 2019.
- [30] M. Yu, T. Xie, T. He, P. McDaniel, and Q. K. Burke, "Flow table security in SDN: Adversarial reconnaissance and intelligent attacks," *IEEE/ACM Trans. Netw.*, vol. 29, no. 6, pp. 2793–2806, Dec. 2021.
- [31] S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, and S. Kambhampati, "A survey of moving target defenses for network security," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1909–1941, 3rd Quart., 2020.
- [32] S. Kim et al., "DIVERGENCE: Deep reinforcement learning-based adaptive traffic inspection and moving target defense countermeasure framework," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4834–4846, Dec. 2022.
- [33] Z. Liu et al., "Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur. 21)*, 2021, pp. 3829–3846.
- [34] K. Doshi, Y. Yilmaz, and S. Uludag, "Timely detection and mitigation of stealthy DDoS attacks via IoT networks," *IEEE Trans. Depend. Secure Comput.*, vol. 18, no. 5, pp. 2164–2176, Oct. 2021.
- [35] M. Yue, J. Li, Z. Wu, and M. Wang, "High-potency models of LDoS attack against cubic+ red," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4950–4965, 2021.
- [36] V. D. M. Rios, P. R. Inacio, D. Magoni, and M. M. Freire, "Detection and mitigation of low-rate denial-of-service attacks: A survey," *IEEE Access*, vol. 10, pp. 76648–76668, 2022.
- [37] M. Yue, M. Wang, and Z. Wu, "Low-high burst: A double potency varying-RTT based full-buffer shrew attack model," *IEEE Trans. Depend. Secure Comput.*, vol. 18, no. 5, pp. 2285–2300, Oct. 2021.
- [38] R. Xie et al., "Disrupting the SDN control channel via shared links: Attacks and countermeasures," *IEEE/ACM Trans. Netw.*, vol. 30, no. 5, pp. 2158–2172, Oct. 2022.
- [39] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, "An approach to application-layer DoS detection," *J. Big Data*, vol. 10, no. 1, p. 22, 2023.
- [40] M. Ghaznavi, E. Jalalpour, M. A. Salahuddin, R. Boutaba, D. Migault, and S. Preda, "Content delivery network security: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2166–2190, 4th Quart., 2021.
- [41] N. Tripathi and N. Hubballi, "Slow rate denial of service attacks against HTTP/2 and detection," *Comput. Secur.*, vol. 72, pp. 255–272, Jan. 2018.
- [42] Y. Jia, F. Zhong, A. Alrawais, B. Gong, and X. Cheng, "Flowguard: An intelligent edge defense mechanism against IoT DDoS attacks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9552–9562, Oct. 2020.
- [43] J. Li, L. Lyu, X. Liu, X. Zhang, and X. Lyu, "FLEAM: A federated learning empowered architecture to mitigate DDoS in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4059–4068, Jun. 2022.
- [44] Z. Wu, L. Zhang, and M. Yue, "Low-rate DoS attacks detection based on network multifractal," *IEEE Trans. Depend. Secure Comput.*, vol. 13, no. 5, pp. 559–567, Oct. 2016.
- [45] D. Tang, X. Wang, X. Li, P. Vijayakumar, and N. Kumar, "AKN-FGD: Adaptive Kohonen network based fine-grained detection of LDoS attacks," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 1, pp. 273–287, Feb. 2023.
- [46] D. Tang, Y. Yan, C. Gao, W. Liang, and W. Jin, "LTrFT: Mitigate the low-rate data plane DDoS attack with learning-to-rank enabled flow tables," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3143–3157, 2023.
- [47] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3431–3446.
- [48] N. Tripathi, "Delays have dangerous ends: Slow HTTP/2 dos attacks into the wild and their real-time detection using event sequence analysis," *IEEE Trans. Depend. Secure Comput.*, vol. 21, no. 3, pp. 1244–1256, Jun. 2024.
- [49] S. Debroy et al., "Frequency-minimal utility-maximal moving target defense against DDoS in SDN-based systems," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 890–903, Jun. 2020.
- [50] Y. Zhou, G. Cheng, Y. Zhao, Z. Chen, and S. Jiang, "Toward proactive and efficient DDoS mitigation in IIoT systems: A moving target defense approach," *IEEE Trans. Ind. Informat.*, vol. 18, no. 4, pp. 2734–2744, Apr. 2022.
- [51] T. Zhang et al., "When moving target defense meets attack prediction in digital twins: A convolutional and hierarchical reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3293–3305, Oct. 2023.

- [52] A. B. Osei, S. R. Yeginati, Y. Al Mtawa, and T. Halabi, "Optimized moving target defense against DDoS attacks in IoT networks: When to adapt?" in *Proc. GLOBECOM IEEE Global Commun. Conf.*, 2022, pp. 2782–2787.
- [53] G. Mani et al., "Machine learning based resilience testing of an address randomization cyber defense," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 6, pp. 4853–4867, Dec. 2023.
- [54] K. Toczé, A. J. Fahs, G. Pierre, and S. Nadjm-Tehrani, "VioLinn: Proximity-aware edge placement with dynamic and elastic resource provisioning," *ACM Trans. Internet Things*, vol. 4, no. 1, pp. 1–31, 2023.
- [55] S.-Y. Chang, Y. Park, and B. B. A. Babu, "Fast IP hopping randomization to secure hop-by-hop access in SDN," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 1, pp. 308–320, Mar. 2019.
- [56] X. Xu, H. Hu, Y. Liu, H. Zhang, and D. Chang, "An adaptive IP hopping approach for moving target defense using a light-weight CNN detector," *Secur. Commun. Netw.*, vol. 2021, no. 1, pp. 1–17, 2021, Art. no. 8848473.
- [57] K. Zeitz, M. Cantrell, R. Marchany, and J. Tront, "Changing the game: A micro moving target IPv6 defense for the Internet of Things," *IEEE Wireless Commun. Lett.*, vol. 7, no. 4, pp. 578–581, Aug. 2018.
- [58] R. E. Navas, H. Sandaker, F. Cuppens, N. Cuppens, L. Toutain, and G. Z. Papadopoulos, "IANVS: A moving target defense framework for a resilient Internet of Things," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, 2020, pp. 1–6.
- [59] J. Tan, H. Jin, H. Hu, R. Hu, H. Zhang, and H. Zhang, "WF-MTD: Evolutionary decision method for moving target defense based on Wright-Fisher process," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 6, pp. 4719–4732, Dec. 2023.
- [60] A. H. Anwar, C. A. Kamhoua, N. O. Leslie, and C. Kiekintveld, "HoneyPot allocation for cyber deception under uncertainty," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 3438–3452, Sep. 2022.
- [61] X. Li, L. Dong, L. Xue, and C. Sun, "Hybrid reinforcement learning for optimal control of non-linear switching system," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 9161–9170, Nov. 2023.
- [62] Y. Gao, A. Abate, L. Xie, and K. H. Johansson, "Distributional reachability for Markov decision processes: Theory and applications," *IEEE Trans. Autom. Control*, early access, Dec. 11, 2023, doi: [10.1109/TAC.2023.3341282](https://doi.org/10.1109/TAC.2023.3341282).
- [63] K. Zhong, Z. Yang, G. Xiao, X. Li, W. Yang, and K. Li, "An efficient parallel reinforcement learning approach to cross-layer defense mechanism in industrial control systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2979–2990, Nov. 2022.
- [64] G. Brockman et al., "Openai gym," 2016, *arXiv:1606.01540*.
- [65] R. Poschinger, N. Rodday, R. Labaca-Castro, and G. Dreo Rodosek, "OpenmtD: A framework for efficient network-level MTD evaluation," in *Proc. 7th ACM Workshop Moving Target Defense*, 2020, pp. 31–41.
- [66] N. Bandi, H. Tajbakhsh, and M. Analoui, "FastMove: Fast IP switching moving target defense to mitigate DDoS attacks," in *Proc. IEEE Conf. Depend. Secure Comput. (DSC)*, 2021, pp. 1–7.
- [67] T. Zhang, C. Xu, J. Shen, X. Kuang, and L. A. Grieco, "How to disturb network reconnaissance: A moving target defense approach based on deep reinforcement learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5735–5748, 2023.

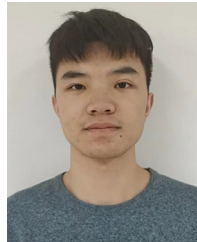


Yuyang Zhou (Member, IEEE) received the B.S. degree in electronic information engineering from the Nanjing University of Science and Technology in 2016, and the Ph.D. degree in cyberspace security from Southeast University in 2021, where he is currently working as a Postdoctoral Fellow with the School of Cyber Science and Engineering. He has published in some of the topmost journals and conferences like IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS,

and ACM CCS, and is involved as a reviewer and a technical program committee of several journals and conferences in the field. His major research interests include moving target defense, DDoS mitigation, and intrusion detection. He is a member of CCF.



Guang Cheng (Member, IEEE) received the B.S. degree in traffic engineering from Southeast University, Nanjing, China, in 1994, the M.S. degree in computer application from the Hefei University of Technology in 2000, and the Ph.D. degree in computer network from Southeast University in 2003, where he is a Full Professor with the School of Cyber Science and Engineering. He has authored or coauthored seven monographs and more than 100 technical papers, including top journals and conferences like IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, and IEEE INFOCOM. His research interests include network security, network measurement, and traffic behavior analysis. He is a Distinguished Member of CCF.



Zhi Ouyang (Student Member, IEEE) received the B.S. degree in information security from the Nanjing University of Posts and Telecommunications in 2021. He is currently pursuing the master's degree with the School of Cyber Science and Engineering, Southeast University. His research interests include moving target defense, cyber deception, and network security.



ZongYao Chen (Student Member, IEEE) received the B.S. degree in information security from Hainan University in 2022. He is currently pursuing the master's degree with the School of Cyber Science and Engineering, Southeast University. His major research interests include moving target defense, android malware detection, and reverse engineering.